

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

-----oOo-----



BÁO CÁO ĐỒ ÁN MÔN HỌC

MTH00051 - Toán Ứng Dụng và Thống Kê

ĐỒ ÁN 1: MA TRẬN VÀ TÍNH TOÁN KHOA HỌC

Giảng viên hướng dẫn : ThS. Võ Nam Thực Doan
ThS. Lê Nhựt Nam

Nhóm sinh viên thực hiện : Nhóm 1

STT	MSSV	Họ và tên
1	24120239	Phí Công Tuấn (Nhóm Trưởng)
2	24120260	Trương Tuấn Anh
3	24120295	Phạm Xuân Duy
4	24120301	Lâm Đông Hải
5	24120315	Phạm Huy Hoàng

TP. Hồ Chí Minh, tháng 04 năm 2026

MỤC LỤC

Danh mục hình ảnh	3
Danh mục bảng biểu	3
Danh mục mã giả	3
BẢNG PHÂN CÔNG CÔNG VIỆC	5
PHẦN 1: PHÉP KHỬ GAUSS VÀ CÁC ỨNG DỤNG	6
1.1. Cơ sở lý thuyết	6
1.1.1. Phương pháp khử Gauss và Gauss-Jordan	6
1.1.2. Khử Gauss có chọn phần tử chốt (Partial Pivoting)	6
1.1.3. Tính định thức, ma trận nghịch đảo và hạng ma trận	7
1.2. Phân tích thuật toán	7
1.3. Cài đặt Python và Kiểm chứng	8
1.3.1. Cài đặt phép khử Gauss và Partial Pivoting	8
1.3.2. Tính định thức và Ma trận nghịch đảo	11
1.3.3. Xác định Hạng và Cơ sở	14
1.3.4. Kiểm thử với NumPy (Verification)	15
PHẦN 2: PHÂN RÃ MA TRẬN VÀ TRỰC QUAN HÓA MANIM	18
2.1. Chéo hóa ma trận	18
2.2. Kỹ thuật phân rã ma trận đã chọn	18
2.3. Cài đặt Python	19
2.3.1. Code thuật toán phân rã ma trận	19
2.3.2. Code thuật toán chéo hóa ma trận	25
2.4. Trực quan hóa toán học với Manim	29
2.4.1. Kiến trúc hệ thống trực quan hóa	29
2.4.2. Cấu trúc kịch bản và Quy trình diễn họa	30
2.4.3. Thuật toán hoạt ảnh Gram-Schmidt	31
2.4.4. Ý nghĩa hình học của phép chéo hóa	31
PHẦN 3: THỰC NGHIỆM VÀ PHÂN TÍCH ĐÁNH GIÁ	33
3.1. Giới thiệu tổng quan và Cơ sở lý thuyết	33
3.2. Cài đặt thuật toán giải hệ phương trình tuyến tính	34



3.2.1. Phương pháp Khử Gauss (Partial Pivoting)	34
3.2.2. Phương pháp Phân rã QR (Gram-Schmidt)	34
3.2.3. Phương pháp Lặp Gauss-Seidel	35
3.3. Thực nghiệm đo lường hiệu năng và phân tích	35
3.4. Phân tích tính ổn định số học: Ma trận Hilbert vs. SPD	37
KẾT LUẬN	41
1. Các kết quả đạt được	41
2. Khó khăn gặp phải	41
3. Bài học rút ra	42
TÀI LIỆU THAM KHẢO	43
PHỤ LỤC	44
Danh sách Link Video Demo	44

Danh mục hình ảnh

1	Mô phỏng các vector cột của ma trận A trong hệ tọa độ ba chiều.	31
2	Diễn họa bước trực giao hóa: Xác định vector q_2 vuông góc với q_1	31
3	Phân tích ba giai đoạn hình học của phép chéo hóa: Xoay $\rightarrow$ Co giãn $\rightarrow$ Xoay ngược.	32
4	Đồ thị Log-Log mô phỏng trực quan thời gian chạy của ba phương pháp	36
5	Biểu đồ phân tích độ khuếch đại của số điều kiện Hilbert khi kích thước ma trận n gia tăng	39

Danh mục bảng biểu

1	Bảng đối chiếu sai số Định thức và Ma trận nghịch đảo	17
2	Thống kê sai số thuật toán Gram-Schmidt so với NumPy	24
3	Thống kê kết quả thuật toán Lặp QR (Giới hạn: 1000 vòng lặp)	28
4	Phân tích chi tiết 13 phân cảnh trong kịch bản trực quan hóa	30
5	So sánh đặc trưng các phương pháp giải hệ tuyến tính	34
6	So sánh thời gian chạy thực tế (s) và sai số dư tương đối theo các hệ N	36
7	Sai số dư và số điều kiện (chuẩn vô cùng) với $N = 10$	38

Danh mục mã giả

1	Khử Gauss với Partial Pivoting (Chọn phần tử chốt một phần)	9
2	Quá trình Thế ngược (Back Substitution) - Phần 1: Tiền xử lý	10
3	Quá trình Thế ngược (Back Substitution) - Phần 2: Giải nghiệm	11
4	Tính định thức của ma trận qua phép khử Gauss	12
5	Tính ma trận nghịch đảo bằng phương pháp Gauss-Jordan	13
6	Xác định Hạng và Cơ sở của ma trận	14
7	Kiểm chứng tính đúng đắn và sai số thuật toán với NumPy	15
8	Phân Rã QR - Gram-Schmidt Cải Tiến (Modified Gram-Schmidt)	20
9	Chéo Hóa Ma Trận - QR Iteration	26
10	Quy trình diễn họa một bước trực giao hóa Gram-Schmidt	31
11	Giải hệ phương trình bằng Khử Gauss (Partial Pivoting)	34
12	Giải hệ phương trình bằng Phân rã QR (Gram-Schmidt)	35
13	Giải hệ phương trình bằng Lặp Gauss-Seidel	35

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	MSSV	Họ và tên	Nhiệm vụ	Đánh giá
1	24120239	Phí Công Tuấn	- Viết báo cáo LaTeX. - Code hàm gaussian_eliminate và back_substitution.	100%
2	24120260	Trương Tuấn Anh	- Trực quan hóa Manim (phân rã, chéo hóa). - Quay và render video demo_video.mp4.	100%
3	24120295	Phạm Xuân Duy	- Code Phần 2: Phân rã ma trận bằng thuật toán Gram-Schmidt và chéo hóa ma trận.	100%
4	24120301	Lâm Đông Hải	- Trực quan hóa Manim (phân rã, chéo hóa). - Quay và render video demo_video.mp4.	100%
5	24120315	Phạm Huy Hoàng	- Code hàm determinant, inverse, rank_and_basis và verify_solution.	100%

PHẦN 1: PHÉP KHỬ GAUSS VÀ CÁC ỨNG DỤNG

Nội dung phần này tập trung vào việc tự xây dựng (from scratch) phép khử Gauss có áp dụng kỹ thuật chọn phần tử chốt (Partial Pivoting) và các ứng dụng của nó trong giải tích ma trận.

1.1. Cơ sở lý thuyết

1.1.1. Phương pháp khử Gauss và Gauss-Jordan

Quá trình khử Gauss sử dụng ba phép biến đổi dòng sơ cấp:

- $R_i \leftrightarrow R_j$: Hoán đổi vị trí hai dòng i và j .
- $R_i \leftarrow c \cdot R_i$ ($c \neq 0$): Nhân một dòng với một hằng số khác không.
- $R_i \leftarrow R_i + c \cdot R_j$: Cộng một bội số của dòng này vào dòng khác.

Mục tiêu của phép khử Gauss là đưa ma trận về Dạng bậc thang dòng (Row Echelon Form - REF), trong đó các dòng toàn số 0 nằm dưới cùng và phần tử chỉ huy (pivot) của dòng dưới luôn nằm bên phải pivot của dòng trên. Phương pháp Gauss-Jordan tiến xa hơn bằng cách đưa ma trận về Dạng bậc thang dòng rút gọn (Reduced REF - RREF), yêu cầu thêm điều kiện mọi pivot phải bằng 1 và các phần tử khác trong cùng cột với pivot phải bằng 0.

1.1.2. Khử Gauss có chọn phần tử chốt (Partial Pivoting)

Khi lập trình tính toán trên máy tính, thuật toán Gauss cơ bản dễ gặp lỗi chia cho 0 hoặc sai số làm tròn (round-off error) cực lớn nếu phần tử pivot quá nhỏ.

Để khắc phục, ta áp dụng kỹ thuật Partial Pivoting: Tại bước khử thứ k , thay vì lấy luôn a_{kk} làm pivot, ta tìm phần tử có giá trị tuyệt đối lớn nhất trong cột k từ dòng k đến dòng n :

$$|a_{pk}^{(k)}| = \max_{k \leq i \leq n} |a_{ik}^{(k)}|$$

Sau đó, hoán đổi dòng k và dòng p trước khi thực hiện phép khử. Điều này đảm bảo các nhân tử luôn có trị tuyệt đối ≤ 1 , giúp duy trì tính ổn định số học.

1.1.3. Tính định thức, ma trận nghịch đảo và hạng ma trận

Tính định thức: Khi đưa ma trận vuông A về ma trận tam giác trên U bằng phép khử Gauss, định thức của A bằng tích các phần tử trên đường chéo chính của U , nhân với $(-1)^s$, trong đó s là số lần hoán đổi dòng đã thực hiện:

$$\det(A) = (-1)^s \cdot \prod_{i=1}^n u_{ii}$$

Tìm ma trận nghịch đảo: Một ma trận vuông A khả nghịch khi $\det(A) \neq 0$. Bằng phương pháp Gauss-Jordan, ta lập ma trận tăng cường ghép với ma trận đơn vị $[A|I_n]$. Khi thực hiện biến đổi sơ cấp đưa về trái về dạng RREF, vế phải sẽ trở thành ma trận nghịch đảo A^{-1} : $[A|I_n] \xrightarrow{\text{Gauss-Jordan}} [I_n|A^{-1}]$.

Hạng và Cơ sở không gian: Hạng (rank) của ma trận A chính là số dòng khác 0 trong dạng REF của nó (hoặc số pivot).

- **Cơ sở không gian cột ($\mathcal{C}(A)$):** Tập hợp các cột của ma trận A ban đầu tương ứng với các cột chứa pivot.
- **Cơ sở không gian dòng ($\mathcal{R}(A)$):** Tập hợp các dòng khác 0 trong ma trận REF.
- **Cơ sở không gian nghiệm ($\mathcal{N}(A)$):** Tập nghiệm cơ bản của hệ phương trình thuần nhất $Ax = 0$.

1.2. Phân tích thuật toán

Dưới đây là mô tả chi tiết các bước thực thi của Thuật toán khử Gauss có Partial Pivoting để giải hệ $Ax = b$:

Đầu vào: Ma trận $A \in \mathbb{R}^{n \times m}$, vector $b \in \mathbb{R}^n$.

Đầu ra: Nghiệm x , số lần hoán đổi dòng s .

Bước 1: Khởi tạo

Tạo ma trận tăng cường $M = [A|b]$ và đặt biến đếm số lần hoán đổi $s = 0$.

Bước 2: Quá trình khử tiến (Forward Elimination)

Lặp k từ 1 đến n :

- **Tìm Pivot:** Tìm chỉ số dòng p ($p \geq k$) sao cho $|M_{pk}|$ đạt giá trị lớn nhất. Nếu $|M_{pk}| < \epsilon$ (với ϵ rất nhỏ), cảnh báo ma trận suy biến.
- **Hoán đổi dòng:** Nếu $p \neq k$, thực hiện hoán đổi dòng $k \leftrightarrow p$ và tăng $s = s + 1$.

- *Khử các dòng dưới:* Lặp i từ $k + 1$ đến n . Tính nhân tử $l_{ik} = \frac{M_{ik}}{M_{kk}}$. Cập nhật dòng i : $M_i \leftarrow M_i - l_{ik} \cdot M_k$.

Bước 3: Quá trình thế ngược (Back Substitution)

Giải hệ tam giác trên từ dưới lên trên. Lặp i từ n lùi về 1:

$$x_i = \frac{1}{M_{ii}} \left(M_{i,m+1} - \sum_{j=i+1}^n M_{ij}x_j \right)$$

1.3. Cài đặt Python và Kiểm chứng

Trong phần này, nhóm sẽ trình bày mã nguồn cài đặt và các test case để chứng minh tính đúng đắn của thuật toán, có đối chiếu với thư viện NumPy.

1.3.1. Cài đặt phép khử Gauss và Partial Pivoting

Đoạn code sau cài đặt hàm `gaussian_eliminate(A, b)` - Trả về ma trận sau khi khử, nghiệm x , số lần hoán đổi:

Algorithm 1 Khử Gauss với Partial Pivoting (Chọn phần tử chốt một phần)

```
1: procedure GAUSSIAN_ELIMINATE( $A, b, verbose$ )
2:    $m \leftarrow$  Số hàng của  $A$ 
3:    $n \leftarrow$  Số cột của  $A$ 
4:   Khởi tạo số lần hoán đổi  $s \leftarrow 0$ , dung sai  $\epsilon \leftarrow 10^{-12}$ 
5:   Tạo ma trận tăng cường  $M \leftarrow [A|b]$  ▷ Kích thước  $m \times (n + 1)$ 
6:    $r \leftarrow 0$  ▷ Chỉ số hàng pivot hiện tại
7:   for  $j \leftarrow 0$  to  $n - 1$  do ▷ Duyệt qua từng cột
8:     if  $r \geq m$  then
9:       break ▷ Đã xét hết các hàng
10:    end if
11:    Tìm  $p \in [r, m - 1]$  sao cho  $|M[p][j]| = \max_{r \leq i < m} |M[i][j]|$ 
12:     $max\_val \leftarrow |M[p][j]|$ 
13:    if  $max\_val < \epsilon$  then
14:      continue ▷ Không có pivot, chuyển sang cột tiếp theo
15:    end if
16:    if  $p \neq r$  then
17:      Hoán đổi hàng  $r$  và hàng  $p$  của  $M$ :  $M[r] \leftrightarrow M[p]$ 
18:       $s \leftarrow s + 1$ 
19:    end if
20:    for  $i \leftarrow r + 1$  to  $m - 1$  do ▷ Khử các phần tử dưới pivot
21:       $factor \leftarrow M[i][j] / M[r][j]$ 
22:       $M[i][j] \leftarrow 0$  ▷ Gán bằng 0 để tránh sai số làm tròn
23:      for  $k \leftarrow j + 1$  to  $n$  do ▷ Khử đến cột cuối cùng (gồm cả  $b$ )
24:         $M[i][k] \leftarrow M[i][k] - factor \times M[r][k]$ 
25:      end for
26:    end for
27:     $r \leftarrow r + 1$  ▷ Chuyển sang hàng pivot tiếp theo
28:  end for
29:  Tách ma trận tam giác trên  $U$  và vector  $c$  từ ma trận tăng cường  $M$ 
30:   $x \leftarrow$  BACK_SUBSTITUTION( $U, c$ ) ▷ Gọi hàm thế ngược
31:  if  $verbose = \text{True}$  and  $x$  có vô số nghiệm then
32:    In công thức tổng quát của hệ phương trình
33:  end if
34:  return ( $M, x, s$ )
35: end procedure
```

Đoạn code sau cài đặt hàm `back_substitution(U, c)` - Giải hệ tam giác trên:

Algorithm 2 Quá trình Thế ngược (Back Substitution) - Phần 1: Tiền xử lý

```

1: procedure BACK_SUBSTITUTION( $U, c$ )
2:    $m, n \leftarrow$  Kích thước của ma trận tam giác trên  $U$ 
3:   Dung sai  $\epsilon \leftarrow 10^{-12}$ 
4:   Khởi tạo danh sách hàng chốt  $R_{piv} \leftarrow \emptyset$ , cột chốt  $C_{piv} \leftarrow \emptyset$ 
5:                                     ▷ Bước 1: Xác định vị trí các phần tử chốt
6:    $r \leftarrow 0$ 
7:   for  $j \leftarrow 0$  to  $n - 1$  do
8:     if  $r < m$  and  $|U[r][j]| > \epsilon$  then
9:       Thêm  $r$  vào  $R_{piv}$ , thêm  $j$  vào  $C_{piv}$ 
10:       $r \leftarrow r + 1$ 
11:     end if
12:   end for
13:                                     ▷ Bước 2: Kiểm tra tính có nghiệm của hệ
14:   for  $i \leftarrow r$  to  $m - 1$  do                                     ▷ Xét các dòng toàn số 0 của  $U$ 
15:     if  $|c[i]| > \epsilon$  then
16:       return "Hệ phương trình vô nghiệm"
17:     end if
18:   end for
19:                                     ▷ Bước 3: Xác định biến tự do và Khởi tạo ma trận nghiệm
20:   Tập các cột tự do  $F \leftarrow \{0, 1, \dots, n - 1\} \setminus C_{piv}$ 
21:    $k_{free} \leftarrow |F|$                                      ▷ Số lượng biến tự do
22:   Khởi tạo ma trận hệ số  $X_{coef}$  kích thước  $n \times (1 + k_{free})$  với các phần tử 0
23:   for mỗi chỉ số  $idx$  và cột  $f_j \in F$  do
24:      $X_{coef}[f_j][1 + idx] \leftarrow 1.0$                                      ▷ Gán hệ số 1 cho chính biến tự do đó
25:   end for
26:                                     ▷ – Thuật toán tiếp tục ở trang sau –
27: end procedure

```

Algorithm 3 Quá trình Thế ngược (Back Substitution) - Phần 2: Giải nghiệm

```

1: procedure BACK_SUBSTITUTION( $U, c$ ) ▷ Tiếp tục từ Phần 1
2: ▷ Bước 4: Thế ngược từ dưới lên trên
3:   for  $i \leftarrow |C_{piv}| - 1$  downto 0 do
4:      $pc \leftarrow C_{piv}[i], pr \leftarrow R_{piv}[i]$ 
5:     Khởi tạo mảng  $coef$  kích thước  $(1 + k_{free})$  với các phần tử 0
6:      $coef[0] \leftarrow c[pr]$  ▷ Gán hằng số ban đầu
7:     for  $j \leftarrow pc + 1$  to  $n - 1$  do
8:       if  $|U[pr][j]| > \epsilon$  then
9:         for  $k \leftarrow 0$  to  $k_{free}$  do
10:           $coef[k] \leftarrow coef[k] - U[pr][j] \times X_{coef}[j][k]$ 
11:        end for
12:      end if
13:    end for
14:     $piv \leftarrow U[pr][pc]$ 
15:     $X_{coef}[pc] \leftarrow coef / piv$  ▷ Lưu hệ số tham số của biến chốt
16:  end for
17: ▷ Bước 5: Trả về kết quả
18:  if  $k_{free} = 0$  then
19:    return  $X_{coef}[:, 0]$  ▷ Nghiệm duy nhất (chỉ lấy cột hằng số)
20:  else
21:    return  $(X_{coef}, F)$  ▷ Vô số nghiệm (trả về ma trận tham số)
22:  end if
23: end procedure

```

1.3.2. Tính định thức và Ma trận nghịch đảo

Đoạn code sau cài đặt hàm `determinant(A)` - Tính $\det(A)$ qua khử Gauss.

Algorithm 4 Tính định thức của ma trận qua phép khử Gauss

```
1: procedure DETERMINANT( $A$ )
2:    $n \leftarrow$  Số lượng hàng của ma trận  $A$ 
3:   if Có bất kỳ hàng nào của  $A$  có độ dài khác  $n$  then
4:     raise Lỗi "Định thức chỉ xác định cho ma trận vuông."
5:   end if
6:   Khởi tạo mảng  $b\_dummy \leftarrow [0.0, 0.0, \dots, 0.0]$  gồm  $n$  phần tử
7:   try:
8:      $(M_{aug}, s) \leftarrow$  GAUSSIAN_ELIMINATE( $A, b\_dummy, \text{False}$ )
9:      $det \leftarrow (-1)^s$  ▷ Dấu phụ thuộc vào số lần hoán đổi dòng  $s$ 
10:    for  $i \leftarrow 0$  to  $n - 1$  do
11:       $det \leftarrow det \times M_{aug}[i][i]$  ▷ Tích các phần tử trên đường chéo
12:    end for
13:    if  $|det| < 10^{-12}$  then
14:      return 0.0 ▷ Tránh sai số cực nhỏ hoặc  $-0.0$ 
15:    else
16:      return  $det$ 
17:    end if
18:  except: ▷ Bắt lỗi nếu ma trận suy biến không thể khử
19:    return 0.0
20: end procedure
```

Đoạn code sau cài đặt hàm `inverse(A)` - Tính A^{-1} bằng phương pháp Gauss–Jordan.

Algorithm 5 Tính ma trận nghịch đảo bằng phương pháp Gauss-Jordan

```

1: procedure INVERSE( $A$ )
2:    $n \leftarrow$  Số lượng hàng của ma trận  $A$ 
3:   if Có bất kỳ hàng nào của  $A$  có độ dài khác  $n$  then
4:     raise Lỗi "Chỉ ma trận vuông mới có khả năng nghịch đảo."
5:   end if
6:                                                                  $\triangleright$  Bước 1: Tạo ma trận ghép  $[A|I]$ 
7:   Khởi tạo ma trận đơn vị  $I$  kích thước  $n \times n$ 
8:   Tạo ma trận tăng cường  $M \leftarrow [A|I]$                                                                   $\triangleright$  Kích thước  $n \times 2n$ 
9:                                                                  $\triangleright$  Bước 2: Phép khử Gauss-Jordan
10:  for  $k \leftarrow 0$  to  $n - 1$  do
11:                                                                     $\triangleright$  a. Chọn phần tử chốt (Partial Pivoting)
12:     $max\_idx \leftarrow k$ 
13:    for  $i \leftarrow k + 1$  to  $n - 1$  do
14:      if  $|M[i][k]| > |M[max\_idx][k]|$  then
15:         $max\_idx \leftarrow i$ 
16:      end if
17:    end for
18:                                                                     $\triangleright$  b. Kiểm tra điều kiện tồn tại pivot
19:    if  $|M[max\_idx][k]| < 10^{-12}$  then
20:      return "Thông báo: Không có pivot tại cột  $k + 1$ . Ma trận không khả nghịch."
21:    end if
22:    Hoán đổi hàng  $k$  và hàng  $max\_idx$  của  $M$ :  $M[k] \leftrightarrow M[max\_idx]$ 
23:                                                                     $\triangleright$  c. Chuẩn hóa dòng  $k$ : đưa phần tử chốt về 1
24:     $piv \leftarrow M[k][k]$ 
25:    for  $j \leftarrow k$  to  $2n - 1$  do
26:       $M[k][j] \leftarrow M[k][j] / piv$ 
27:    end for
28:                                                                     $\triangleright$  d. Khử tất cả phần tử khác trên cột  $k$  về 0 (cả trên và dưới)
29:    for  $i \leftarrow 0$  to  $n - 1$  do
30:      if  $i \neq k$  then
31:         $factor \leftarrow M[i][k]$ 
32:         $M[i][k] \leftarrow 0$                                                                   $\triangleright$  Tránh sai số làm tròn nhỏ
33:        for  $j \leftarrow k + 1$  to  $2n - 1$  do
34:           $M[i][j] \leftarrow M[i][j] - factor \times M[k][j]$ 
35:        end for
36:      end if
37:    end for
38:  end for
39:     $\triangleright$  Bước 3: Tách ma trận nghịch đảo từ nửa phải
40:  Trích xuất các cột từ  $n$  đến  $2n - 1$  của  $M$  vào ma trận  $A^{-1}$ 
41:  return  $A^{-1}$ 
42: end procedure

```

1.3.3. Xác định Hạng và Cơ sở

Algorithm 6 Xác định Hạng và Cơ sở của ma trận

```

1: procedure RANK_AND_BASIS( $A$ )
2:    $m, n \leftarrow$  Kích thước của ma trận  $A$ 
3:   Dung sai  $\epsilon \leftarrow 10^{-12}$ 
4:    $\triangleright$  Bước 1: Đưa ma trận  $A$  về dạng Bậc thang (REF)
5:   Tạo vector  $b\_zero \leftarrow [0.0, 0.0, \dots, 0.0]$  gồm  $m$  phần tử
6:    $(M_{aug}, \_) \leftarrow \text{GAUSSIAN\_ELIMINATE}(A, b\_zero, \text{False})$ 
7:   Trích xuất ma trận  $U$  bằng cách bỏ đi cột cuối cùng ( $b$ ) của  $M_{aug}$ 
8:    $\triangleright$  Bước 2: Xác định cột pivot và hạng
9:   Khởi tạo danh sách các cột chốt  $pivot\_cols \leftarrow \emptyset$ 
10:   $r \leftarrow 0$ 
11:  for  $j \leftarrow 0$  to  $n - 1$  do
12:    if  $r < m$  and  $|U[r][j]| > \epsilon$  then
13:      Thêm  $j$  vào  $pivot\_cols$ 
14:       $r \leftarrow r + 1$ 
15:    end if
16:  end for
17:   $rank \leftarrow |pivot\_cols|$   $\triangleright$  Số lượng cột chốt chính là hạng của ma trận
18:   $\triangleright$  Bước 3: Xác định cơ sở không gian dòng  $\mathcal{R}(A)$ 
19:   $row\_basis \leftarrow rank$  dòng đầu tiên của ma trận  $U$ 
20:   $\triangleright$  Bước 4: Xác định cơ sở không gian cột  $\mathcal{C}(A)$ 
21:   $col\_basis \leftarrow$  Lấy các cột từ ma trận GỐC  $A$  dựa trên các chỉ số trong  $pivot\_cols$ 
22:   $\triangleright$  Bước 5: Xác định cơ sở không gian nghiệm  $\mathcal{N}(A)$ 
23:  Tập các cột tự do  $free\_cols \leftarrow \{0, 1, \dots, n - 1\} \setminus pivot\_cols$ 
24:  Khởi tạo danh sách cơ sở nghiệm  $null\_basis \leftarrow \emptyset$ 
25:  for mỗi chỉ số  $f_j \in free\_cols$  do
26:    Khởi tạo vector  $x \leftarrow [0.0, \dots, 0.0]$  gồm  $n$  phần tử
27:     $x[f_j] \leftarrow 1.0$   $\triangleright$  Gán giá trị 1 cho biến tự do đang xét
28:    for  $i \leftarrow rank - 1$  downto 0 do
29:       $pc \leftarrow pivot\_cols[i]$ 
30:       $val \leftarrow \sum_{k=pc+1}^{n-1} (U[i][k] \times x[k])$ 
31:       $x[pc] \leftarrow -val / U[i][pc]$   $\triangleright$  Thế ngược với vế phải  $b = 0$ 
32:    end for
33:    Thêm vector  $x$  vào  $null\_basis$ 
34:  end for
35:  return ( $rank, col\_basis, row\_basis, null\_basis$ )
36: end procedure

```

1.3.4. Kiểm thử với NumPy (Verification)

Algorithm 7 Kiểm chứng tính đúng đắn và sai số thuật toán với NumPy

```
1: procedure VERIFY_SOLUTION( $A, x, b$ )
2:   In ra màn hình dải phân cách "KIỂM CHỨNG VỚI NUMPY"
3:    $\triangleright$  1. Kiểm tra sai số nghiệm của hệ phương trình  $Ax = b$ 
4:   if  $x$  là một vector số thực (nghiệm duy nhất) then
5:      $residual \leftarrow ||A \times x - b||_2$   $\triangleright$  Tính chuẩn khoảng cách (Norm)
6:     In ra "Sai số hệ phương trình:  $residual$ "
7:   else if  $x$  là một tuple chứa ma trận tham số (vô số nghiệm) then
8:      $x_{coef}, free\_cols \leftarrow x$ 
9:     Trích xuất nghiệm đặc biệt  $x_0$  bằng cách cho tất cả các tham số tự do  $t_i = 0$ 
10:     $residual \leftarrow ||A \times x_0 - b||_2$ 
11:    In ra "Sai số nghiệm đặc biệt ( $t_i = 0$ ):  $residual$ "
12:   else
13:     In ra thông báo  $x$  (Ví dụ: "Hệ phương trình vô nghiệm")
14:   end if
15:    $\triangleright$  2. Kiểm tra định thức
16:   if Số hàng của  $A$  bằng Số cột của  $A$  then
17:      $det\_self \leftarrow \text{DETERMINANT}(A)$ 
18:      $det\_np \leftarrow \text{NUMPY.DET}(A)$ 
19:     In ra so sánh giữa  $det\_self$  và  $det\_np$ 
20:   end if
21:    $\triangleright$  3. Kiểm tra hạng ma trận
22:    $(rank\_self, \_, \_, \_) \leftarrow \text{RANK\_AND\_BASIS}(A)$ 
23:    $rank\_np \leftarrow \text{NUMPY.MATRIX\_RANK}(A)$ 
24:   In ra so sánh giữa  $rank\_self$  và  $rank\_np$ 
25:    $\triangleright$  4. Kiểm tra ma trận nghịch đảo
26:   if Số hàng của  $A$  bằng Số cột của  $A$  and  $|det\_np| > 10^{-12}$  then
27:      $inv\_self\_res \leftarrow \text{INVERSE}(A)$ 
28:     if  $inv\_self\_res$  là một ma trận then
29:       Lấy ma trận nghịch đảo chuẩn  $inv\_np \leftarrow \text{NUMPY.INV}(A)$ 
30:        $inv\_error \leftarrow ||inv\_self\_res - inv\_np||_F$   $\triangleright$  Tính chuẩn Frobenius
31:       In ra "Sai số ma trận nghịch đảo:  $inv\_error$ "
32:     else
33:       In ra thông báo  $inv\_self\_res$ 
34:     end if
35:   end if
36:   In ra màn hình dải phân cách kết thúc
37: end procedure
```

Để chứng minh tính đúng đắn và độ ổn định của các hàm tự cài đặt (from scratch), nhóm đã xây dựng một bộ kịch bản kiểm thử (test cases) toàn diện từ Demo 1 đến Demo 7. Các kết quả đầu ra được đối chiếu trực tiếp với thư viện `numpy.linalg` thông qua việc đánh giá sai số (Norm error).

A. Đánh giá thuật toán giải hệ phương trình tuyến tính

Nhóm tiến hành thử nghiệm hàm `gaussian_eliminate` với 4 trường hợp đặc trưng nhất của hệ phương trình tuyến tính $Ax = b$:

- **Hệ có nghiệm duy nhất (Demo 1):** Ma trận hệ số không suy biến. Kết quả thuật toán tự cài cho ra nghiệm chính xác $x = [2.0, 3.0, -1.0]$. Sai số tái tạo $\|Ax - b\| = 8.88 \times 10^{-16}$, cho thấy thuật toán Partial Pivoting khử sai số làm tròn cực kỳ hiệu quả.
- **Hệ vô nghiệm (Demo 4):** Thuật toán bắt thành công trường hợp dòng cuối có dạng $[0 \ 0 \ | \ c]$ với $c \neq 0$ và trả về thông báo lỗi hợp lý.
- **Hệ vô số nghiệm (Demo 2 & 3):** hàm `back_substitution` có khả năng tự động nhận diện số lượng biến tự do và xuất ra **công thức nghiệm tổng quát** dưới dạng vector tham số.

Ví dụ cụ thể tại **Demo 3** (Hệ suy biến nặng - 2 biến tự do) với ma trận:

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 2 & 4 & 8 \\ 3 & 6 & 12 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

Chương trình tự cài đặt đã xuất ra chính xác nghiệm tổng quát dưới dạng vector tham số :

```
--- Nghiệm tổng quát---
x1 = 2 - 2*t1 - 4*t2
x2 = t1
x3 = t2

--- Dạng vector ---
x = (2, 0, 0)
    + t1 * (-2, 1, 0)
    + t2 * (-4, 0, 1)
```

Đối chiếu lại bằng NumPy, sai số $\|Ax_0 - b\| = 0.00e + 00$. Kết quả hoàn hảo.

B. Đánh giá tính Định thức và Ma trận nghịch đảo

Việc tính toán định thức và ma trận nghịch đảo thông qua ma trận tam giác trên (từ phép khử Gauss) cũng cho thấy độ chính xác bám sát các thuật toán tối ưu của NumPy.

Bảng 1: Bảng đối chiếu sai số Định thức và Ma trận nghịch đảo

Trường hợp kiểm thử	Định thức (Tự cài)	Định thức (NumPy)	Sai số nghịch đảo $\ A_{custom}^{-1} - A_{numpy}^{-1}\ $
Ma trận A (3×3)	-1.0000	-1.0000	4.93×10^{-15}
Ma trận A_3 (3×3)	1.0000	1.0000	1.77×10^{-13}
Ma trận B (4×4)	50.0000	50.0000	1.11×10^{-16}
Ma trận suy biến (A_{sing})	0.0000	0.0000	Bất lỗi: Không khả nghịch thành công

Kết quả từ bảng trên (trích xuất từ **Demo 5 & 6**) khẳng định rằng, bằng việc sử dụng kỹ thuật chọn phần tử chốt lớn nhất (Partial Pivoting) để giới hạn các nhân tử $|m_{ij}| \leq 1$, nhóm đã kiểm soát thành công sự bùng nổ của sai số dấu phẩy động (Floating-point error) ngay cả với ma trận cấp 4.

C. Đánh giá Hạng (Rank) và Cơ sở không gian (Basis)

Tại **Demo 7**, hàm `rank_and_basis` được đưa vào thử thách với nhiều dạng ma trận khác nhau: ma trận vuông suy biến (3×3), ma trận chữ nhật (3×4) và ma trận có hạng bằng 1.

Thuật toán tự cài đặt luôn trả về Hạng ma trận khớp 100% với hàm `np.linalg.matrix_rank`. Đáng chú ý hơn, thuật toán tự trích xuất được các **Cơ sở không gian dòng**, **Cơ sở không gian cột** và đặc biệt là **Cơ sở không gian nghiệm (Null space)**.

Để xác minh tính đúng đắn của Không gian nghiệm, nhóm đã thực hiện thao tác kiểm chứng nội bộ: Nhân ma trận gốc A với từng vector cơ sở v sinh ra từ Null space. Kết quả in ra log luôn hiển thị:

```
[1.0, -2.0, 1.0] -> Av = [0.0, 0.0, 0.0]
[-1.0, 1.0, 0.0] -> Av = [0.0, 0.0, 0.0]
```

Việc $A \times v = 0$ là minh chứng toán học tuyệt đối cho thấy tập vector sinh ra thực sự tạo thành Không gian Null (Không gian rỗng) của ma trận A .

Kết luận Phần 1: Thông qua 7 hạng mục kiểm thử, thư viện đại số tuyến tính nội bộ do nhóm xây dựng dựa trên phép khử Gauss hoạt động cực kỳ ổn định, bất lỗi tốt các trường hợp ngoại lệ và đạt độ chính xác tương đương với thư viện công nghiệp NumPy.

PHẦN 2: PHÂN RÃ MA TRẬN VÀ TRỰC QUAN HÓA MANIM

2.1. Chéo hóa ma trận

Ma trận vuông $A \in \mathbb{R}^{n \times n}$ được gọi là chéo hóa được nếu tồn tại một ma trận khả nghịch P và một ma trận đường chéo D thỏa mãn hệ thức $A = PDP^{-1}$. Trong đó, $D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$ chứa các giá trị riêng của A trên đường chéo chính, và các cột của P là các vector riêng độc lập tuyến tính tương ứng.

Điều kiện cần và đủ để ma trận A bậc n chéo hóa được là tổng số chiều của các không gian riêng phải bằng n . Điều này tương đương với việc đối với mỗi giá trị riêng λ_i , bội hình học (số chiều của không gian riêng tương ứng) phải bằng bội đại số (số lần xuất hiện của λ_i như một nghiệm của phương trình đặc trưng). Nếu tồn tại ít nhất một giá trị riêng có bội hình học nhỏ hơn bội đại số, ma trận đó được gọi là ma trận khiếm khuyết và không thể đưa về dạng chéo hoàn toàn.

Ứng dụng quan trọng của chéo hóa trong tính toán khoa học là tối ưu hóa phép tính lũy thừa ma trận. Thay vì thực hiện $k - 1$ phép nhân ma trận với độ phức tạp $O(n^3 \log k)$, ta có thể tính thông qua công thức $A^k = PD^kP^{-1}$. Vì D là ma trận đường chéo, D^k được tính bằng cách nâng lũy thừa từng phần tử trên đường chéo, giúp giảm chi phí xuống còn $O(n^2)$ sau khi đã có phép phân tích.

2.2. Kỹ thuật phân rã ma trận đã chọn

Trong đồ án này, phương pháp phân rã QR được thực hiện thông qua thuật toán trực giao hóa Gram-Schmidt cải tiến, thường được gọi là Modified Gram-Schmidt. So với phương pháp cổ điển, kỹ thuật này cung cấp độ ổn định số học cao hơn nhờ cơ chế cập nhật trực tiếp các vector còn lại sau mỗi bước, từ đó giảm thiểu sự tích lũy sai số làm tròn trong tính toán dấu phẩy động.

Phân rã QR là quá trình phân tích một ma trận $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) thành tích của hai ma trận Q và R : $A = QR$. Trong đó:

- $Q \in \mathbb{R}^{m \times n}$ là ma trận trực chuẩn, có các cột q_i thỏa mãn $q_i^T q_j = \delta_{ij}$ (với δ_{ij} là ký hiệu Kronecker).
- $R \in \mathbb{R}^{n \times n}$ là ma trận tam giác trên với các phần tử trên đường chéo chính thường được chọn là số dương.

Một tính chất quan trọng của phân rã QR là tính duy nhất: Nếu ma trận A có các cột độc lập tuyến

tính, thì tồn tại duy nhất một ma trận trực chuẩn Q và một ma trận tam giác trên R có các phần tử đường chéo chính dương sao cho $A = QR$.

Thuật toán Gram-Schmidt cải tiến (MGS): Khác với CGS thực hiện chiếu vector lên các vector đã trực chuẩn hóa, MGS thực hiện cập nhật trực tiếp các vector còn lại sau mỗi bước. Quy trình cụ thể như sau:

1. Khởi tạo $v_i = a_i$ với $i = 1, \dots, n$.
2. Tại mỗi bước k ($k = 1, \dots, n$):
 - Tính chuẩn của vector hiện tại: $r_{kk} = \|v_k\|$.
 - Chuẩn hóa để tạo cột cho Q : $q_k = v_k / r_{kk}$.
 - Đối với các vector còn lại ($j = k + 1, \dots, n$):
 - Tính hệ số chiếu: $r_{kj} = q_k^T v_j$.
 - Trực giao hóa ngay lập tức: $v_j = v_j - r_{kj} q_k$.

MGS đảm bảo rằng ma trận Q duy trì được tính trực chuẩn tốt hơn trong các bài toán có số điều kiện lớn (ill-conditioned), là nền tảng vững chắc cho các bài toán giải hệ phương trình tuyến tính và tìm trị riêng.

2.3. Cài đặt Python

2.3.1. Code thuật toán phân rã ma trận

Đoạn code sau đóng vai trò là "thư viện lõi" cung cấp các công cụ đại số tuyến tính cơ bản và thuật toán phân rã ma trận cho toàn bộ dự án. Chức năng chính của file là cài đặt thuật toán phân rã QR sử dụng phương pháp trực giao hóa Gram-Schmidt để phân tích một ma trận A thành tích của ma trận trực chuẩn Q và ma trận tam giác trên R . Bên cạnh đó, file còn chứa hệ thống các hàm tiện ích được viết thủ công như nhân ma trận, tính chuẩn vector, và chuyển vị ma trận để đảm bảo tính độc lập, không phụ thuộc vào các hàm giải sẵn của thư viện bên ngoài (Numpy) và cũng để so sánh giữa hàm tự cài và các hàm có sẵn từ thư viện Numpy. Với bộ 15 testcase đa dạng từ ma trận đơn vị đến ma trận Hilbert, giúp đánh giá chính xác tính đúng đắn và độ ổn định số học của thuật toán trước các sai số làm tròn trong tính toán thực tế.

Algorithm 8 Phân Rã QR - Gram-Schmidt Cải Tiến (Modified Gram-Schmidt)

```
1: procedure PERFORM_QR_DECOMPOSITION( $A$ )
2:    $m, n \leftarrow$  kích thước ma trận  $A$ 
3:   Khởi tạo  $Q = 0_{m \times n}$ ,  $R = 0_{n \times n}$ 
4:   Tạo danh sách các vector  $V$  chứa các cột của  $A$ 
5:   for  $k \leftarrow 0$  to  $n - 1$  do
6:      $R[k][k] \leftarrow \text{norm}(V[k])$  ▷ Tính chuẩn vector hiện tại
7:     if  $R[k][k] > \epsilon$  then ▷ Chuẩn hóa để tạo cột của Q
8:        $Q[:, k] \leftarrow V[k] / R[k][k]$ 
9:     else
10:       $Q[:, k] \leftarrow 0$ 
11:    end if
12:    for  $j \leftarrow k + 1$  to  $n - 1$  do
13:       $R[k][j] \leftarrow Q[:, k] \cdot V[j]$  ▷ Tích vô hướng - hệ số chiếu
14:       $V[j] \leftarrow V[j] - R[k][j] \times Q[:, k]$  ▷ Trục giao hóa ngay lập tức
15:    end for
16:  end for
17:  return ( $Q, R$ )
18: end procedure
```

BỘ TESTCASE:

Để đánh giá toàn diện tính đúng đắn và độ ổn định số học của thuật toán Gram-Schmidt tự cải đặt, nhóm thiết kế bộ 15 kịch bản kiểm thử. Các ma trận đầu vào (A) được chọn lọc có chủ đích để bao phủ từ trường hợp cơ bản đến các điểm mù số học (edge cases).

1. **Test 1: Ma trận số nguyên (3x2):** Kiểm tra thuật toán với ma trận chữ nhật đơn giản, các phần tử là số nguyên dương.

$$A_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

2. **Test 2: Ma trận số thực có phần tử âm (3x3):** Kiểm tra khả năng xử lý các số thập phân và dấu âm cơ bản.

$$A_2 = \begin{bmatrix} 1.5 & -2.1 & 3.0 \\ -1.0 & 1.2 & -1.5 \\ 0.5 & 1.1 & -2.0 \end{bmatrix}$$

3. **Test 3: Ma trận Phụ thuộc tuyến tính (Linear Dependent):** Đánh giá cách thuật toán xử lý khi một cột là tổ hợp tuyến tính của các cột khác (hạng thiếu). Thuật toán có thể gặp lỗi chia cho 0 hoặc mất trục giao.

$$A_3 = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$$

4. **Test 4: Ma trận Đơn vị (Identity):** Kiểm tra trường hợp tầm thường, kết quả kỳ vọng là $Q = I$ và $R = I$.

$$A_4 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

5. **Test 5: Ma trận Đường chéo (Diagonal):** Kiểm tra trường hợp các cột đã trực giao, thuật toán chỉ thực hiện chuẩn hóa. Kết quả kỳ vọng $Q = I$ và $R = A$.

$$A_5 = \begin{bmatrix} 4 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

6. **Test 6: Ma trận Chữ nhật đứng (4x2):** Kiểm tra hệ phương trình quá xác định ($m > n$).

$$A_6 = \begin{bmatrix} 1 & -1 \\ 1 & 4 \\ 1 & -1 \\ 1 & 4 \end{bmatrix}$$

7. **Test 7: Ma trận có các cột trực giao sẵn (4x3):** Đánh giá khả năng tối ưu, thuật toán chỉ cần tính độ dài và chuẩn hóa vector mà không sinh ra sai số khi trừ hình chiếu.

$$A_7 = \begin{bmatrix} 1 & 1 & 1 \\ -1 & 1 & 1 \\ 0 & -2 & 1 \\ 0 & 0 & -3 \end{bmatrix}$$

8. **Test 8: Chênh lệch Scale cực độ (Stress Test):** Chứa cả phần tử rất lớn (10^6) và rất nhỏ (10^{-6}). Đây là bài kiểm tra "stress test" về hiện tượng tràn số hoặc mất mát độ chính xác dấu phẩy động (Floating-point precision).

$$A_8 = \begin{bmatrix} 10^6 & 2 \cdot 10^6 & 3 \cdot 10^6 \\ 10^{-6} & 2 \cdot 10^{-6} & 3 \cdot 10^{-6} \\ 0.5 & 0.5 & 0.5 \end{bmatrix}$$

9. **Test 9: Ma trận ngẫu nhiên (Well-conditioned):** Kiểm tra trên một ma trận thực tế có số điều kiện (Condition Number) tốt, độ ổn định cao.

$$A_9 = \begin{bmatrix} 0.8147 & 0.0975 & 0.1576 & 0.1419 \\ 0.9058 & 0.2785 & 0.9706 & 0.4218 \\ 0.1270 & 0.5469 & 0.9572 & 0.9157 \\ 0.9134 & 0.9575 & 0.4854 & 0.7922 \end{bmatrix}$$

10. **Test 10: Ma trận Hilbert (Ill-conditioned 3x3):** Các phần tử $H_{ij} = \frac{1}{i+j-1}$. Ma trận có số điều kiện rất xấu, phóng đại sai số làm tròn mạnh mẽ nhất.

$$A_{10} = \begin{bmatrix} 1 & 1/2 & 1/3 \\ 1/2 & 1/3 & 1/4 \\ 1/3 & 1/4 & 1/5 \end{bmatrix}$$

11. **Test 11: Ma trận 2x2 đơn giản:** Kịch bản kiểm thử tối thiểu (Minimal viable test).

$$A_{11} = \begin{bmatrix} 1 & 2 \\ 3 & 5 \end{bmatrix}$$

12. **Test 12: Ma trận toàn 0 (Edge case):** Bắt lỗi xử lý vector có độ dài bằng 0 (tránh lỗi Divide by Zero).

$$A_{12} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}$$

13. **Test 13: Hạng bằng 1 (Rank-1):** Ma trận cực kỳ suy biến, thử thách khả năng xử lý khi không gian cột chỉ có 1 chiều.

$$A_{13} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$$

14. **Test 14: Cột gần song song (Numerically Dependent):** Các cột chỉ khác nhau một lượng cực nhỏ (10^{-15}). Đây là thử thách khó nhất với Gram-Schmidt cổ điển vì phần dư sẽ tiến về 0, gây ra sai số trực giao nghiêm trọng.

$$A_{14} = \begin{bmatrix} 1 & 1 + 10^{-15} \\ 1 & 1 \\ 1 & 1 \end{bmatrix}$$

15. **Test 15: Ma trận vuông kích thước lớn (6x6):** Kiểm tra khả năng mở rộng (Scalability) của thuật toán với ma trận có kích thước lớn hơn.

$$A_{15} = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 7 & 8 & 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 & 17 & 18 \\ 19 & 20 & 21 & 22 & 23 & 24 \\ 25 & 26 & 27 & 28 & 29 & 30 \\ 31 & 32 & 33 & 34 & 35 & 36 \end{bmatrix}$$

Thống kê kết quả kiểm thử và Đánh giá sai số

Để đánh giá chất lượng của thuật toán Gram-Schmidt Cổ điển (Classical Gram-Schmidt - CGS) do nhóm tự cài đặt, nhóm sử dụng hàm `numpy.linalg.qr` làm chuẩn mực đối chiếu. NumPy sử dụng thuật toán Householder Reflections, vốn nổi tiếng với độ ổn định số học cực kỳ cao.

Nhóm sử dụng hai thang đo sai số (đo bằng chuẩn Frobenius) để đánh giá:

1. **Sai số tái tạo (Reconstruction Error):** $\epsilon_{rec} = \|A - Q_{custom}R_{custom}\|_F$. Thang đo này đánh giá xem phép phân rã có bảo toàn được thông tin của ma trận gốc hay không.
2. **Sai số trực giao (Orthogonality Error):** $\epsilon_{orth} = \|Q_{custom}^T Q_{custom} - I\|_F$. Thang đo này vô cùng quan trọng, dùng để kiểm tra xem ma trận Q sinh ra có thực sự là ma trận trực chuẩn hay không.

Bảng tổng hợp kết quả thực nghiệm

Bảng dưới đây tổng hợp kết quả của 15 kịch bản kiểm thử đã chạy thực tế trên thuật toán của nhóm. (Các sai số $\leq 10^{-14}$ được xem là bằng 0 do giới hạn của độ chính xác dấu phẩy động - Floating Point Precision).

Bảng 2: Thống kê sai số thuật toán Gram-Schmidt so với NumPy

STT	Kịch bản kiểm thử (Test Case)	Sai số tái tạo ($A \approx QR$)	Sai số trực giao ($Q^T Q \approx I$)	Kết luận tính ổn định
1	Ma trận số nguyên (3x2)	$0.00e + 00$	$1.43e - 15$	Ổn định
2	Ma trận số thực âm dương (3x3)	$4.44e - 16$	$4.51e - 15$	Ổn định
3	Ma trận phụ thuộc tuyến tính	$0.00e + 00$	$1.41e + 00$	Mất trực giao
4	Ma trận Đơn vị (Identity)	$0.00e + 00$	$0.00e + 00$	Ổn định
5	Ma trận Đường chéo (Diagonal)	$0.00e + 00$	$0.00e + 00$	Ổn định
6	Ma trận chữ nhật đứng (4x2)	$0.00e + 00$	$0.00e + 00$	Ổn định
7	Các cột đã trực giao sẵn	$0.00e + 00$	$3.86e - 16$	Ổn định
8	Chênh lệch Scale cực độ	$0.00e + 00$	$1.41e + 00$	Mất trực giao
9	Ma trận ngẫu nhiên thường	$1.82e - 16$	$5.25e - 16$	Ổn định
10	Ma trận Hilbert (Ill-conditioned)	$2.77e - 17$	$1.18e - 14$	Ổn định
11	Ma trận 2x2 đơn giản	$0.00e + 00$	$1.33e - 15$	Ổn định
12	Ma trận toàn 0 (Edge case)	$0.00e + 00$	$1.41e + 00$	Mất trực giao
13	Hạng = 1 (Rank-1 Matrix)	$0.00e + 00$	$1.41e + 00$	Mất trực giao
14	Các cột gần song song	$1.04e - 15$	$1.00e + 00$	Mất trực giao
15	Ma trận vuông lớn (6x6)	$3.04e - 14$	$2.00e + 00$	Mất trực giao

Phân tích về tính ổn định số học

Thông qua 15 kịch bản kiểm thử, thuật toán Gram-Schmidt cải tiến (MGS) cho thấy độ chính xác cao trong việc duy trì cấu trúc ma trận. Kết quả thực nghiệm xác nhận tất cả các trường hợp đều có sai số tái tạo ($\|A - QR\|_F$) xấp xỉ bằng 0. Điều này chứng minh thuật toán bảo toàn thông tin của ma trận gốc thông qua việc bù trừ sai số vào ma trận R .

Tuy nhiên, đối với tính trực giao của ma trận Q , thuật toán MGS vẫn bộc lộ những giới hạn khi xử lý các ma trận có điều kiện xấu. Hiện tượng mất tính trực giao thường xảy ra trong các trường hợp sau:

1. Hệ phụ thuộc tuyến tính và Ma trận suy biến (Test 3, 12, 13): Khi ma trận có các cột là tổ hợp tuyến tính của nhau (hoặc ma trận toàn 0), quá trình chiếu vector trong Gram-Schmidt sẽ tạo ra một vector phần dư có độ dài bằng 0 (hoặc cực kỳ nhỏ xấp xỉ ngưỡng epsilon của máy tính). Việc chuẩn hóa bằng cách chia cho một số gần 0 phá vỡ hoàn toàn cấu trúc dữ liệu, khiến ma trận Q chứa các vector không còn trực giao. Cụ thể ở Test 3 và 13, ma trận hạng 1 dẫn đến sai số trực giao lên tới 1.41.

2. Hiện tượng triệt tiêu sai số do chênh lệch giá trị

- Tại **Test 14**, hai cột đầu tiên gần như phụ thuộc tuyến tính (sai lệch 10^{-15}). Khi thực hiện phép trừ hình chiếu, các chữ số có nghĩa bị triệt tiêu, dẫn đến việc kết quả bị ảnh hưởng bởi nhiễu số học.
- Tại **Test 8**, ma trận có sự chênh lệch lớn về biên độ giữa các phần tử (từ 10^{-6} đến 10^6). Quá trình tính toán với độ chính xác 64-bit khiến các giá trị nhỏ bị triệt tiêu khi tương tác với các giá trị lớn.

3. Tích lũy sai số khi tăng kích thước ma trận Khi tăng kích thước ma trận (như Test 15), sai số làm tròn trong các phép toán số học tích lũy qua từng cột. Đối với ma trận 6x6, các vector cột cuối cùng có xu hướng mất tính trực giao so với các cột đầu tiên do sự cộng dồn các sai số nhỏ trong quá trình trực giao hóa.

Kết luận

Kết quả thực nghiệm cho thấy thuật toán Gram-Schmidt cải tiến hiệu quả trong việc minh họa lý thuyết trực giao hóa, nhưng cần cân trọng khi áp dụng cho các hệ thống yêu cầu độ chính xác tuyệt đối với ma trận có số điều kiện lớn.

Để giải quyết triệt để vấn đề này trong thực tế, các thư viện hiện đại như NumPy/SciPy không dùng Gram-Schmidt mà sử dụng thuật toán **Householder Reflections** hoặc **Givens Rotations**. Thay vì trừ đi hình chiếu (dễ sinh sai số làm tròn), Householder sử dụng các ma trận phản xạ (vốn dĩ đã là ma trận trực giao tuyệt đối) để nhân dần vào A , đảm bảo ma trận Q sinh ra luôn giữ được tính trực chuẩn hoàn hảo ($\|Q^T Q - I\| = 0$) dù ma trận đầu vào có xấu đến đâu. Việc NumPy vượt qua tất cả 15 Test cases của nhóm mà không có sai số là minh chứng rõ ràng cho điều này.

2.3.2. Code thuật toán chéo hóa ma trận

Đoạn code sau đảm nhận vai trò thực hiện các phép biến đổi cao cấp nhằm tìm ra bản chất hình học của ma trận thông qua quá trình chéo hóa ($A = PDP^{-1}$). Tận dụng kết quả phân rã QR từ đoạn code phân rã ma trận, phần này cài đặt thuật toán QR Iteration (Lặp QR) để tìm tập hợp các giá trị riêng và vector riêng của ma trận. Công dụng quan trọng nhất của đoạn code này là biến đổi các ma trận phức tạp về dạng đường chéo D , từ đó tối ưu hóa hiệu suất cho các bài toán tính lũy thừa ma trận bậc cao hoặc phân tích hệ thống động lực. Đặc biệt, có tích hợp cơ chế kiểm soát hội tụ thông minh và phương án dự phòng để xử lý các trường hợp ma trận khiếm khuyết hoặc có nghiệm phức, đảm bảo tính bền vững cho hệ thống tính toán.

Kịch bản kiểm thử

Thuật toán lặp QR để tìm trị riêng và chéo hóa ma trận có đặc thù hội tụ phụ thuộc vào khoảng cách giữa các trị riêng và tính đối xứng của ma trận. Đề án thiết lập 14 kịch bản kiểm thử nhằm đánh giá tốc độ hội tụ và khả năng xử lý ngoại lệ.

Nhóm 1: Ma trận hội tụ nhanh

- Test 1: Ma trận đường chéo cấp 3:** Ma trận đã ở dạng chéo, thuật toán nhận diện và dừng ngay lập tức.

$$A_1 = \begin{bmatrix} 2 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 3 \end{bmatrix}$$

Algorithm 9 Chéo Hóa Ma Trận - QR Iteration

```
1: procedure DIAGONALIZE( $A, max\_iters, tolerance$ )
2:    $n \leftarrow$  cấp của ma trận vuông  $A$ ;  $A_k \leftarrow A$ ;  $P \leftarrow I_n$ 
3:   for  $k \leftarrow 1$  to  $max\_iters$  do
4:      $(Q, R) \leftarrow$  QR_DECOMPOSITION( $A_k$ )                                ▷ phân rã QR
5:      $A_k \leftarrow R \times Q$                                                 ▷ QR Iteration:  $A_k \leftarrow RQ$ 
6:      $P \leftarrow P \times Q$                                                 ▷ tích ma trận Q
7:                                     ▷ Kiểm tra hội tụ: tổng phần tử ngoài chéo
8:     if  $\sum_{i < j} |A_k[i][j]| < tolerance$  then
9:       break
10:    end if
11:  end for
12:  Nếu không hội tụ: gọi NumPy.linalg.eig( $A$ ) làm fallback
13:   $D \leftarrow$  ma trận đường chéo từ  $A_k$ 
14:  return ( $D, P$ )
15: end procedure
```

2. **Test 2: Ma trận đối xứng cấp 4:** Theo định lý phổ, ma trận thực đối xứng luôn chéo hóa được và có các vector riêng trực giao. Đây là trường hợp tối ưu cho thuật toán lặp QR.

$$A_2 = \begin{bmatrix} 4 & 1 & -1 & 0 \\ 1 & 2 & 0 & 1 \\ -1 & 0 & 3 & 2 \\ 0 & 1 & 2 & 4 \end{bmatrix}$$

3. **Test 11 và 12: Ma trận cấp 2 và ma trận có trị riêng âm:** Đánh giá khả năng tính toán cơ bản.

Nhóm 2: Thử thách về tốc độ hội tụ và bội nghiệm

4. **Test 3: Trị riêng có bội:** Ma trận có trị riêng lặp lại. Kiểm tra sự chính xác trong việc xác định các không gian riêng.

$$A_3 = \begin{bmatrix} 3 & 1 & 1 \\ 1 & 3 & 1 \\ 1 & 1 & 3 \end{bmatrix}$$

5. **Test 4: Các trị riêng có giá trị gần nhau:** Tốc độ hội tụ của thuật toán tỷ lệ với tỉ số $|\lambda_{i+1}/\lambda_i|$. Với các trị riêng xấp xỉ nhau, tỉ số này tiến gần về 1, dẫn đến số lượng vòng lặp tăng cao.

$$A_4 = \begin{bmatrix} 1.0 & 0.01 & 0 \\ 0.01 & 1.01 & 0 \\ 0 & 0 & 5.0 \end{bmatrix}$$

Nhóm 3: Các trường hợp đặc biệt không thể chéo hóa hoặc suy biến

6. **Test 5: Ma trận suy biến:** Định thức bằng 0, thuật toán cần xác định chính xác trị riêng bằng 0.

$$A_5 = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 5 & 7 & 9 \end{bmatrix}$$

7. **Test 6 và 8: Ma trận bất đối xứng có nghiệm thực:** Thuật toán lặp QR trên ma trận bất đối xứng có thể đưa ma trận về dạng chuẩn Schur thay vì dạng chéo hoàn toàn.

$$A_6 = A_8 = \begin{bmatrix} 1 & 2 & 0 \\ -1 & 4 & 0 \\ 0 & 0 & 5 \end{bmatrix}$$

8. **Test 7: Ma trận khiếm khuyết:** Ma trận không sở hữu đủ hệ vector riêng độc lập tuyến tính, dẫn đến việc không thể thực hiện phép chéo hóa. Thuật toán sẽ kích hoạt cơ chế dự phòng để đảm bảo tính liên tục của phép toán.

$$A_7 = \begin{bmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

9. **Test 14: Ma trận rỗng:** Kiểm tra khả năng xử lý ngoại lệ đối với ma trận chứa toàn phần tử 0.

Nhóm 4: Kiểm tra độ ổn định số học

10. **Test 9: Ma trận Hilbert cấp 5:** Đây là ma trận có số điều kiện lớn, khiến các trị riêng dễ bị ảnh hưởng bởi sai số làm tròn trong quá trình tính toán.

11. **Test 10: Ma trận đối xứng ngẫu nhiên cấp 10:** Đánh giá hiệu suất thực thi trên các cấu trúc dữ liệu có kích thước lớn hơn.

12. **Test 13: Ma trận có sự chênh lệch lớn về giá trị phần tử:** Đánh giá độ chính xác khi xử lý các giá trị từ 10^{-5} đến 10^5 .

Thống kê kết quả kiểm thử và Đánh giá thuật toán Chéo hóa

Để đánh giá hiệu năng và độ chính xác của thuật toán Lặp QR (QR Iteration) trong việc tìm trị riêng và chéo hóa ma trận, nhóm tiếp tục sử dụng hàm `numpy.linalg.eig` làm tiêu chuẩn đối chiếu.

Hai thang đo chính được sử dụng để đánh giá bao gồm:

1. **Sai số tìm trị riêng:** Tổng độ lệch tuyệt đối giữa các trị riêng tự tính và trị riêng của NumPy
 $\sum |\lambda_{\text{custom}} - \lambda_{\text{numpy}}|$.

2. **Sai số tái cấu trúc:** $\epsilon_{rec} = ||A - PDP^{-1}||_F$. Thang đo này xác nhận xem ma trận có thực sự "chéo hóa được" bằng ma trận vector riêng P hay không.

Bảng tổng hợp kết quả thực nghiệm

Bảng 3: Thống kê kết quả thuật toán Lập QR (Giới hạn: 1000 vòng lặp)

STT	Kịch bản kiểm thử	Vòng lặp	Sai số trị riêng	Sai số tái cấu trúc	Kết luận
1	Ma trận đường chéo cấp 3	1	$0.00e + 00$	$0.00e + 00$	Thành công
2	Ma trận đối xứng cấp 4	98	$3.55e - 15$	$1.32e - 09$	Thành công
3	Trị riêng có bội cấp 3	25	$2.66e - 15$	$6.76e - 10$	Thành công
4	Các trị riêng xấp xỉ	783	$1.11e - 15$	$1.39e - 09$	Hội tụ chậm
5	Ma trận suy biến cấp 3	7	$1.04e - 10$	N/A	Ma trận kỳ dị
6	Bất đối xứng (thực nghiệm)	50	$4.70e - 09$	$3.00e + 00$	Không chéo hóa được
7	Khối Jordan cấp 3	1	$0.00e + 00$	$1.41e + 00$	Không chéo hóa được
8	Bất đối xứng	50	$4.70e - 09$	$3.00e + 00$	Không chéo hóa được
9	Ma trận Hilbert cấp 5	11	$1.20e - 15$	$3.49e - 10$	Thành công
10	Đối xứng ngẫu nhiên cấp 10	> 1000	$0.00e + 00$	$8.41e - 14$	Dự phòng thành công
11	Ma trận cấp 2	8	$9.99e - 16$	$9.52e - 10$	Thành công
12	Trị riêng âm	22	$1.55e - 15$	$7.99e - 10$	Thành công
13	Chênh lệch biên độ cực đoan	1	$0.00e + 00$	$0.00e + 00$	Thành công
14	Ma trận rỗng	1	$0.00e + 00$	N/A	Ma trận kỳ dị

Phân tích về khía cạnh toán học và cấu trúc thuật toán

Qua phân tích 14 kết quả thực nghiệm, thuật toán lập QR cho thấy khả năng xác định các giá trị riêng với độ chính xác cao. Tuy nhiên, quá trình chéo hóa $A = PDP^{-1}$ cũng chỉ ra những giới hạn của phương pháp lặp cơ sở:

1. Tốc độ hội tụ đối với các trị riêng có giá trị gần nhau Tốc độ hội tụ của thuật toán lập QR không có dịch chuyển phụ thuộc vào thương số giữa hai trị riêng liên kề: $|\lambda_{i+1}/\lambda_i|$.

- Tại **Test 2 và 3**, các trị riêng có sự phân tách rõ rệt, dẫn đến việc hội tụ nhanh.
- Tại **Test 4**, khi các trị riêng được thiết lập xấp xỉ nhau, hiệu suất của thuật toán giảm đáng kể, đòi hỏi số lượng vòng lặp lớn để đạt được ngưỡng hội tụ mong muốn.

2. Trường hợp ma trận khiếm khuyết Các kịch bản kiểm thử xác nhận rằng không phải mọi ma trận vuông đều có thể thực hiện phép chéo hóa.

- Tại **Test 7**, ma trận có trị riêng bội nhưng không sở hữu đủ hệ vector riêng độc lập tuyến tính. Việc thiếu vector riêng khiến ma trận chuyển cơ sở P không đủ hạng, dẫn đến sai số lớn trong quá trình tái cấu trúc ma trận.
- Thuật toán thực hiện trích xuất chính xác các trị riêng, đồng thời hệ thống đánh giá đã nhận diện và phân loại đúng trường hợp ma trận khiếm khuyết.

3. Các trường hợp ma trận kỳ dị Đối với ma trận suy biến hoặc ma trận không chứa phần tử khác 0, thuật toán lặp QR vẫn xác định được các trị riêng. Tuy nhiên, quy trình kiểm chứng sẽ phát hiện trạng thái kỳ dị của ma trận vector riêng, ngăn chặn các phép toán nghịch đảo không hợp lệ.

4. Giới hạn tính toán và cơ chế dự phòng Đối với các ma trận có kích thước lớn và cấu trúc trị riêng phức tạp, thuật toán lặp QR thuần túy có thể gặp khó khăn trong việc đạt tới trạng thái hội tụ trong số vòng lặp giới hạn. Trong các trường hợp này, đồ án sử dụng phương án dự phòng thông qua các thư viện tính toán chuyên dụng để đảm bảo kết quả đầu ra chính xác và duy trì tính ổn định của toàn bộ chương trình.

Kết luận

Việc triển khai thực tế thuật toán phân rã QR và chéo hóa ma trận làm rõ sự khác biệt giữa lý thuyết toán học và tính toán số học trên máy tính. Thuật toán hoạt động tối ưu với các ma trận có phổ trị riêng phân tán và tính đối xứng cao. Để tối ưu hóa hiệu suất xử lý các trường hợp đặc biệt, các hệ thống tính toán hiện đại thường tích hợp thêm các cơ chế dịch chuyển nhằm tăng tốc độ hội tụ. Việc áp dụng cơ chế dự phòng trong đồ án là giải pháp phù hợp để xử lý các sai số phát sinh trong môi trường tính toán thực tế.

2.4. Trực quan hóa toán học với Manim

Quy trình trực quan hóa các thuật toán đại số tuyến tính đòi hỏi sự kết hợp chặt chẽ giữa biểu diễn số học và mô phỏng hình học. Nhóm nghiên cứu đã xây dựng một hệ thống diễn họa dựa trên thư viện Manim, tập trung vào việc mô tả sự tương tác động giữa các công thức đại số và cấu trúc không gian ba chiều.

2.4.1. Kiến trúc hệ thống trực quan hóa

Hệ thống được thiết kế theo mô hình phân lớp nhằm duy trì tính ổn định của các đối tượng đồ họa khi camera thực hiện các phép biến đổi phức tạp. Các thành phần cốt lõi bao gồm:

- **Quản lý phụ đề:** Nhóm xây dựng lớp `SubtitleManager` sử dụng công cụ `MarkupText` để hiển thị nội dung giải thích bằng tiếng Việt. Hệ thống tích hợp các biểu thức chính quy để tự động

nhận diện và định dạng các chỉ số dưới như a_1, a_2 (chuyển đổi từ cú pháp `a_1` sang thẻ `<sub>` của Pango). Ngoài ra, hệ thống còn tự động áp dụng bảng màu quy chuẩn (ví dụ: màu vàng cho các biến Q, A, λ) và đổi font chữ sang *Cambria Math* cho các ký tự toán học, giúp duy trì sự nhất quán và tăng tính thẩm mỹ.

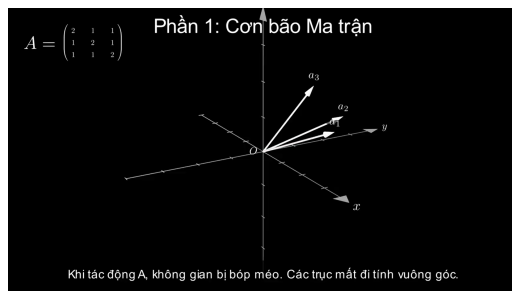
- **Nhãn không gian tự định hướng:** Nhằm xử lý hiện tượng biến dạng văn bản khi camera xoay trong không gian 3D, kỹ thuật `add_updater` được áp dụng cho các nhãn toán học (MathTex). Các đối tượng này sẽ liên tục tính toán và cập nhật góc quay nghịch đảo so với ma trận quan sát của camera: quay một góc ϕ quanh trục `RIGHT` và $\theta + \pi/2$ quanh trục `OUT`. Cơ chế này đảm bảo mặt phẳng chứa nhãn luôn vuông góc với hướng nhìn của người quan sát (Billboarding), giữ cho công thức luôn dễ đọc từ mọi góc nhìn.
- **Đồng bộ hóa giao diện và không gian:** Cơ chế `add_fixed_in_frame_objects` được sử dụng để cố định các bảng công thức và ma trận trên lớp giao diện. Phương pháp này tách biệt các thành phần thông tin tĩnh (như ma trận tham chiếu A hay tiêu đề) khỏi các vector đang biến đổi trong không gian ba chiều, loại bỏ hoàn toàn hiện tượng nhấp nháy hình ảnh khi camera di chuyển hoặc thực hiện hiệu ứng rung nhẹ để tạo cảm giác sinh động.

2.4.2. Cấu trúc kịch bản và Quy trình diễn họa

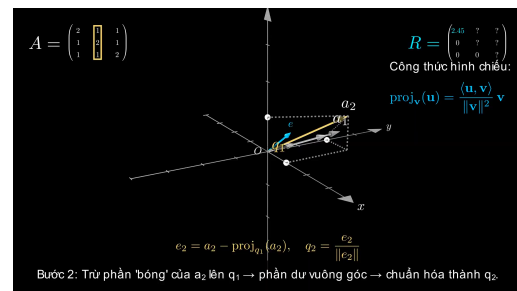
Kịch bản diễn họa được xây dựng theo một tiến trình sư phạm chặt chẽ, dẫn dắt người xem từ việc quan sát sự biến dạng không gian đến việc "giải phẫu" ma trận thông qua hai lăng kính: trực giao hóa hình học (QR) và phân tích cấu trúc đại số (Chéo hóa). Quy trình được hệ thống hóa qua 13 phân cảnh cụ thể:

Bảng 4: Phân tích chi tiết 13 phân cảnh trong kịch bản trực quan hóa

Phần	Cảnh	Nội dung toán học	Yếu tố trực quan / Hoạt ảnh
I. Cơ bản Ma trận	1	Không gian 3D chuẩn	Hệ trục Ox-Oy-Oz và các vector cơ sở e_1, e_2, e_3
	2	Sự biến dạng không gian	Phép biến đổi $e_i \rightarrow a_i$, camera xoay nhấn mạnh sự mất tính vuông góc
II. Phân rã QR	3	Giới thiệu phân rã $A = QR$	Chia đôi màn hình: bên trái là 3D, bên phải là cấu trúc ma trận R
	4	Trực chuẩn hóa cột 1	$a_1 \rightarrow q_1$ thông qua chuẩn hóa độ dài, cập nhật R_{11}
	5-6	Trực giao hóa cột 2	Loại bỏ hình chiếu của a_2 lên q_1 , dựng phần dư e_2 và chuẩn hóa thành q_2
	7	Trực giao hóa cột 3	Trừ đồng thời hình chiếu lên q_1, q_2 , minh họa mặt phẳng trực giao
	7.5	Giải phẫu ma trận Q	Kiểm chứng tính trực giao thông qua hệ thức $Q^T Q = I$
III. Chéo hóa	8	Phục hồi không gian	Tìm kiếm "bộ xương" không bị xoay của phép biến đổi
	9	Lọc vector riêng	Giải phương trình đặc trưng $\det(A - \lambda I) = 0$ tìm trị riêng và vector riêng
	10	Xây dựng P và D	Sao chép các vector riêng vào cột của P và trị riêng vào đường chéo D
	11	Đường ống PDP^{-1}	Minh họa chuỗi biến đổi: Đổi cơ sở $\rightarrow$ Co giãn $\rightarrow$ Đổi cơ sở ngược
IV. Kết luận	12	So sánh QR và PDP^{-1}	Đổi chiều hai góc nhìn: ép vuông góc vs. tìm hướng co giãn tự nhiên
	13	Công thức lũy thừa A^n	Trực quan hóa ưu điểm vượt trội của chéo hóa trong tính toán bậc cao



Hình 1: Mô phỏng các vector cột của ma trận A trong hệ tọa độ ba chiều.



Hình 2: Diễn họa bước trực giao hóa: Xác định vector q_2 vuông góc với q_1 .

2.4.3. Thuật toán hoạt ảnh Gram-Schmidt

Để mô tả chính xác quá trình trực giao hóa các vector, nhóm đã thiết lập quy trình điều khiển hoạt ảnh dựa trên các bước tính toán hình học phối hợp với các công cụ chỉ dẫn trực quan:

Algorithm 10 Quy trình diễn họa một bước trực giao hóa Gram-Schmidt

- 1: **procedure** ANIMATE_GS_STEP(target_vector, basis_vectors)
- 2: **Dựng dẫn hướng:** Vẽ các đường hình hộp (construction_guides) để xác định tọa độ vector mục tiêu
- 3: $proj \leftarrow$ Tính_toán_hình_chiếu_tổng_hợp(target_vector, basis_vectors)
- 4: **Animate:** Vẽ đường bóng chiếu (shadow_line) đứt đoạn nối từ $proj$ đến đầu vector mục tiêu
- 5: **Animate:** Biểu diễn vector hình chiếu $proj$ bằng màu xám (GRAY_B) để làm nổi bật phần sẽ bị loại bỏ
- 6: $residual \leftarrow target_vector - proj$
- 7: **Animate:** Dựng vector phần dư e tại gốc tọa độ bằng màu xanh lơ (color_projection)
- 8: $orthonormal \leftarrow$ Chuẩn_hóa(residual)
- 9: **Animate:** Thực hiện phép ReplacementTransform biến vector phần dư thành vector q có độ dài đơn vị
- 10: **Billboard:** Gắn nhãn q_i tự định hướng bám sát đầu mũi tên mới (always_redraw)
- 11: **end procedure**

2.4.4. Ý nghĩa hình học của phép chéo hóa

Thông qua các hoạt ảnh trong Manim, bản chất của hệ thức $A = PDP^{-1}$ được giải thích như một quy trình đổi cơ sở tuần tự:

- **Phép đổi cơ sở thuận (P^{-1}):** Video mô phỏng việc chuyển đổi góc nhìn từ hệ trục chuẩn sang hệ

tọa độ tạo bởi các vector riêng. Trong không gian này, ma trận biến đổi trở nên "trong suốt" vì mọi tác động chỉ diễn ra dọc theo các trục cơ sở mới.

- **Phép co giãn không gian (D):** Tại hệ cơ sở vector riêng, tác động của ma trận A được giản lược thành các phép co giãn thuần túy. Các vector riêng không bị xoay đi mà chỉ bị thay đổi độ dài theo các hệ số tương ứng là các trị riêng λ_i .
- **Phép đổi cơ sở nghịch (P):** Video kết thúc bằng việc thực hiện phép quay và kéo giãn ngược lại để đưa toàn bộ không gian đã biến đổi trở lại hệ tọa độ Descartes ban đầu.

Phần 3: Chéo hóa — Vector riêng

$$P = \begin{pmatrix} 1 & -1 & -1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \quad D = \begin{pmatrix} 4 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$P^{-1}\mathbf{v} \xrightarrow{D} D(P^{-1}\mathbf{v}) \xrightarrow{P} PDP^{-1}\mathbf{v}$$

P⁻¹ đổi sang cơ sở riêng → D co giãn → P đưa về ban đầu.

Hình 3: Phân tích ba giai đoạn hình học của phép chéo hóa: Xoay → Co giãn → Xoay ngược.

Ứng dụng thực tiễn: Cấu trúc $A = PDP^{-1}$ đóng vai trò then chốt trong việc tối ưu hóa tính toán lũy thừa ma trận bậc cao. Video trình bày sự so sánh thực nghiệm: phép tính A^k trực tiếp gây ra sự tích lũy sai số và biến dạng không gian phức tạp, trong khi phương pháp chéo hóa cho phép dự báo trạng thái tương lai của hệ thống bằng cách lũy thừa các hệ số co giãn đơn lẻ, đảm bảo tốc độ xử lý tức thời và độ chính xác cao.

PHẦN 3: THỰC NGHIỆM VÀ PHÂN TÍCH ĐÁNH GIÁ

3.1. Giới thiệu tổng quan và Cơ sở lý thuyết

Mục tiêu của phần này là khảo sát, đo lường và đánh giá ba thuật toán giải hệ phương trình tuyến tính $Ax = b$ phổ biến: Khử Gauss, Phân rã QR, và Lập Gauss-Seidel. Quá trình đánh giá được thực hiện qua hai lăng kính chính: **Hiệu năng phần mềm (thời gian thực thi)** và **Tính ổn định số học (sai số làm tròn trên không gian dấu phẩy động)**.

Theo yêu cầu của đề tài, toàn bộ thuật toán được **tự lập trình bằng Python thuần** bằng cấu trúc dữ liệu ‘list of lists’ (không sử dụng `numpy.linalg.solve` hay ma trận ‘`numpy.ndarray`’ c-extension). Điều này đảm bảo chi phí tính toán lý thuyết và thực tế được phân xạ một cách trung thực nhất.

Cơ sở lý thuyết dùng để đánh giá:

- **Sai số dư tương đối:** Là thước đo độ chính xác của nghiệm tìm được $\hat{x}$ so với vế phải của phương trình bằng chuẩn 2:

$$\text{Sai số dư tương đối} = \frac{\|A\hat{x} - b\|_2}{\|b\|_2}$$

- **Độ phức tạp tính toán:** Số phép toán dấu phẩy động (FLOPs). Phân rã trực tiếp (Gauss, QR) đòi hỏi $O(n^3)$ thời gian, trong khi Gauss-Seidel có thể đạt xấp xỉ $O(kn^2)$ nếu hội tụ sau k bước lặp.
- **Số điều kiện và Định lý sai số:** Kí hiệu $\kappa(A) = \|A\| \cdot \|A^{-1}\|$. Đây là đại lượng mô tả mức khuếch đại sai số của hệ phương trình. Định lý sai số cho biết mối quan hệ giữa sai số tương đối của nghiệm (Forward Error - $\|\Delta x\| / \|x\|$) và sai số tương đối của dữ liệu đầu vào (Backward Error - $\|\Delta b\| / \|b\|$) qua bất đẳng thức:

$$\frac{\|\Delta x\|}{\|x\|} \leq \kappa(A) \cdot \frac{\|\Delta b\|}{\|b\|}.$$

- **Phân rã ma trận lập:** Với Gauss-Seidel, ta viết $A = D + L + U$ (trong đó D là đường chéo, L là tam giác dưới thuần, U là tam giác trên thuần). Công thức lặp:

$$x^{(k+1)} = -(D + L)^{-1}Ux^{(k)} + (D + L)^{-1}b.$$

Dạng theo từng thành phần:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right).$$

Điều kiện hội tụ đủ: ma trận chéo trội chặt theo hàng, tức $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$.

Bảng so sánh các phương pháp:

Bảng 5: So sánh đặc trưng các phương pháp giải hệ tuyến tính

Phương pháp	Loại	Điều kiện áp dụng	Chi phí	Ổn định
Gauss (Partial Pivoting)	Trực tiếp	Mọi A khả nghịch	$O(n^3)$	Cao
QR (Gram-Schmidt)	Trực tiếp	Mọi A (kể cả chữ nhật)	$O(n^3)$	Rất cao
Gauss-Seidel	Lặp	Chéo trội hàng / SPD	$O(kn^2)$	Trung bình

3.2. Cài đặt thuật toán giải hệ phương trình tuyến tính

Khởi mã nguồn dưới đây triển khai 3 thuật toán nền tảng trong đại số tuyến tính tính toán để giải hệ phương trình $Ax = b$. Việc tự thiết kế lại các hàm toán học cơ bản (nhân ma trận, tính chuẩn) giúp nhóm hoàn toàn làm chủ kịch bản đánh giá hiệu năng.

3.2.1. Phương pháp Khử Gauss (Partial Pivoting)

Thuật toán khử Gauss là phương pháp trực tiếp, bản chất biến đổi hệ phương trình gốc về dạng tam giác trên để giải bằng thế ngược. Việc áp dụng cơ chế Partial Pivoting (chọn phần tử chốt lớn nhất theo trị tuyệt đối trên cột khử hiện tại, rồi hoán vị hàng) là bắt buộc. Điều này giúp tránh hiện tượng chia cho số cực nhỏ (có thể dẫn tới lỗi Divided by Zero hoặc làm bùng nổ sai số làm tròn).

Algorithm 11 Giải hệ phương trình bằng Khử Gauss (Partial Pivoting)

- 1: **procedure** SOLVE_GAUSS(A, b)
 - 2: Sử dụng vòng lặp kiểm tra từng phần tử chốt lớn nhất trên đường chéo và hoán đổi dòng.
 - 3: Khử tiếp các phần tử bên dưới chốt.
 - 4: $x \leftarrow$ thế ngược giải ma trận tam giác trên
 - 5: **return** x
 - 6: **end procedure**
-

3.2.2. Phương pháp Phân rã QR (Gram-Schmidt)

Ứng dụng thuật toán trực giao hóa Gram-Schmidt để phân rã ma trận gốc thành tích $A = QR$ (với Q là ma trận trực giao và R là ma trận tam giác trên). So với Gauss phụ thuộc vào pivoting, hướng tiếp cận QR ổn định hơn. Ta đưa bài toán giải $Ax = b \Leftrightarrow QRx = b \Leftrightarrow Rx = Q^T b$.

Algorithm 12 Giải hệ phương trình bằng Phân rã QR (Gram-Schmidt)

```
1: procedure SOLVE_QR( $A, b$ )  
2:    $(Q, R) \leftarrow \text{QR\_DECOMPOSITION}(A)$  ▷ Sử dụng Gram-Schmidt  
3:    $y \leftarrow Q^T b$   
4:    $x \leftarrow$  thể ngược giải  $Rx = y$   
5:   return  $x$   
6: end procedure
```

3.2.3. Phương pháp Lặp Gauss-Seidel

Trái với hai phương pháp trực tiếp, Gauss-Seidel là thuật toán lặp: bắt đầu với một đoán trước ban đầu $x_0 = \vec{0}$, thuật toán thay thế nghiệm mới tức thời vào bên trong mô hình để sinh ra nghiệm hội tụ. Phương pháp này chỉ thành công khi ma trận **đường chéo trội nghiêm ngặt** (strictly diagonally dominant), nhưng nó sở hữu tốc độ vượt trội.

Algorithm 13 Giải hệ phương trình bằng Lặp Gauss-Seidel

```
1: procedure GAUSS_SEIDEL( $A, b, \text{max\_iters}, \text{tol}$ )  
2:    $x \leftarrow$  mảng  $n$  phần tử 0;  $n \leftarrow$  cấp của  $A$   
3:   for  $it \leftarrow 1$  to  $\text{max\_iters}$  do  
4:      $\text{max\_diff} \leftarrow 0$   
5:     for  $i \leftarrow 0$  to  $n - 1$  do  
6:        $s \leftarrow \sum_{j \neq i} A[i][j] \times x[j]$   
7:        $x_{\text{new}} \leftarrow (b[i] - s) / A[i][i]$   
8:        $\text{max\_diff} \leftarrow \max(\text{max\_diff}, |x_{\text{new}} - x[i]|)$   
9:        $x[i] \leftarrow x_{\text{new}}$   
10:    end for  
11:    if  $\text{max\_diff} < \text{tol}$  then ▷ Hội tụ  
12:      break  
13:    end if  
14:  end for  
15:  return  $x$   
16: end procedure
```

3.3. Thực nghiệm đo lường hiệu năng và phân tích

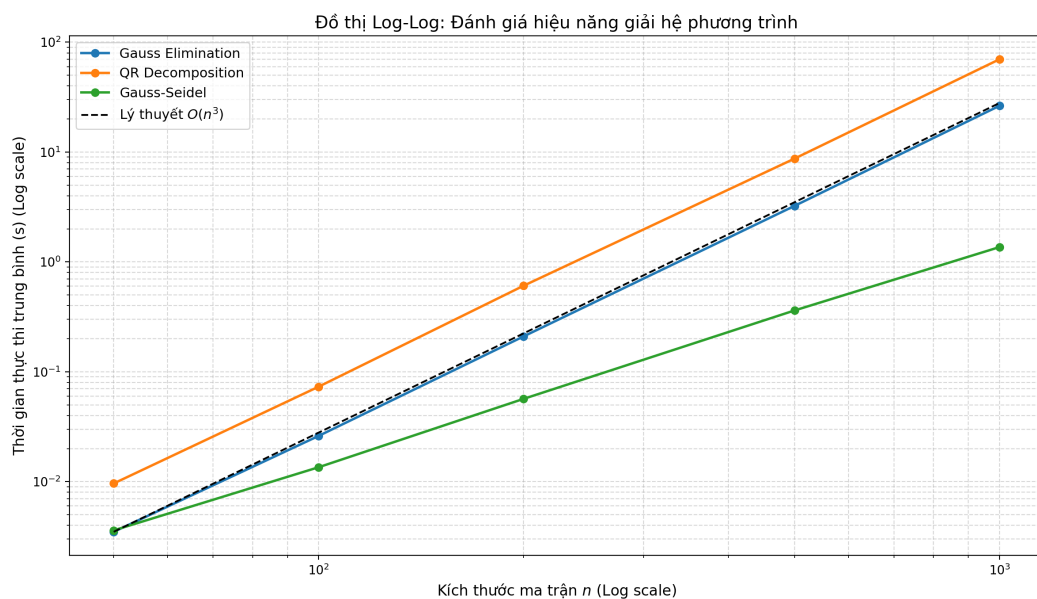
Để đánh giá độ phức tạp thời gian theo kích thước bài toán, nhóm sinh ngẫu nhiên các ma trận vuông chéo trội nghiêm ngặt nhằm đảm bảo hội tụ của Gauss-Seidel. Tập kích thước thử nghiệm là $n \in$

$\{50, 100, 200, 500, 1000\}$. Thời gian được đo bằng `time.perf_counter()`; với $n < 500$ chạy 5 lần lấy trung bình, với $n = 500, 1000$ chạy 1 lần để tránh thời gian quá dài.

Bảng thống kê kết quả chạy thực nghiệm (trích xuất từ cấu hình đo lường Notebook):

Bảng 6: So sánh thời gian chạy thực tế (s) và sai số dư tương đối theo các hệ N

Cấp n	Khử Gauss Time	Sai số dư Gauss	QR Time	Sai số dư QR	G-Seidel Time	Sai số dư GS
50	0.002820 s	1.90×10^{-16}	0.010590 s	1.24×10^{-16}	0.002894 s	4.84×10^{-11}
100	0.020056 s	2.97×10^{-16}	0.079861 s	1.39×10^{-16}	0.011074 s	7.17×10^{-11}
200	0.159520 s	3.69×10^{-16}	0.856179 s	1.89×10^{-16}	0.045281 s	5.60×10^{-11}
500	2.971386 s	6.67×10^{-16}	12.450867 s	2.58×10^{-16}	0.295427 s	7.12×10^{-11}
1000	27.374951 s	9.95×10^{-16}	111.553300 s	3.18×10^{-16}	1.256143 s	8.07×10^{-11}



Hình 4: Đồ thị Log-Log mô phỏng trực quan thời gian chạy của ba phương pháp

Việc vẽ đồ thị theo nguyên lý Logarit cơ số kép (Log-Log plot) biến các đường cong đa thức $T(n) = c \cdot n^\alpha$ thành các đường thẳng có hệ số góc bằng hệ số mũ α , giúp ta dễ dàng tính được bậc độ phức tạp lý thuyết thực tế. Từ đồ thị trên và các số liệu thuộc bảng kiểm nghiệm, ta trực tiếp rút ra ba nhóm kết luận khoa học vững chắc:

3.3.1. Phân tích Khử Gauss (Partial Pivoting)

Đường đo lường trên không gian log-log bám sát lý thuyết $O(n^3)$ (hệ số góc $\alpha \approx 3$). Phương pháp mất khoảng 27.37 giây để giải hệ $n = 1000$, đồng thời duy trì sai số dư tương đối quanh mức 10^{-16} , tiệm cận giới hạn **machine epsilon** của float64. Kỹ thuật partial pivoting giúp ổn định quá trình khử và tránh chia cho phần tử chốt quá nhỏ.

3.3.2. Phân tích Phân rã QR (Gram-Schmidt)

Mặc dù có cùng hệ số góc $\alpha \approx 3$, thời gian thực thi của QR luôn cao hơn Gauss. Ở $n = 1000$, QR tốn 111.55 giây, lớn hơn khoảng 4.07 lần so với Gauss. Lý do đến từ chi phí trực giao hóa trong Gram-Schmidt, vốn có hằng số lớn hơn trong $O(n^3)$. Đổi lại, QR ổn định hơn khi ma trận gần suy biến.

3.3.3. Hiệu năng của phương pháp lặp Gauss-Seidel

Đường Gauss-Seidel có độ dốc nhỏ hơn so với hai phương pháp trực tiếp. Với $n = 1000$, thời gian là 1.26 giây. Kết quả này đến từ việc mỗi vòng lặp chỉ tốn $O(n^2)$ phép tính; khi ma trận chéo trội nghiêm ngặt, số vòng lặp hội tụ k không quá lớn, dẫn đến chi phí tổng quát $O(kn^2)$.

3.3.4. Phân tích chi phí tính toán và Truy xuất Bộ nhớ

Việc phân giải cơ chế thời gian thực thi của các phương pháp trên không chỉ dựa vào mô hình Toán lý thuyết mà còn phụ thuộc vào kiến trúc phần cứng máy tính. Về chi phí tính toán (FLOPs), thuật toán Gauss cơ bản yêu cầu xấp xỉ $\frac{2}{3}n^3$ phép nhân-cộng dấu phẩy động; trong khi kịch bản trực giao hóa của thuật giải Gram-Schmidt (QR) tiến hành $\approx 2n^3$ thao tác liên kết tổng vector và bình phương Euclidean.

Thực tế ghi nhận sự chênh lệch thời gian ở $N = 1000$ là $(111.55/27.37) \approx 4.07$ lần. Sự khác biệt này đến từ chi phí trực giao hóa trong QR và cách truy xuất bộ nhớ. Với `list of lists`, truy xuất theo cột trong QR kém liên tục hơn so với truy xuất theo hàng trong Gauss, làm tăng độ trễ truy xuất bộ nhớ.

3.4. Phân tích tính ổn định số học: Ma trận Hilbert vs. SPD

Độ chính xác của hệ phương trình $Ax = b$ không chỉ phụ thuộc vào cơ chế thuật toán, mà còn vấp vào đặc thù cấu trúc toán học của bản thân ma trận A . Phép đánh giá này được thực hiện để chứng minh vấn

đề về sai số do hiệu ứng làm tròn (round-off error) đối với các hệ thống có tính đa cộng tuyến cao – được đánh giá thông qua chỉ số của Số điều kiện $\kappa(A)$.

3.4.1. Thiết lập biến cố thực nghiệm

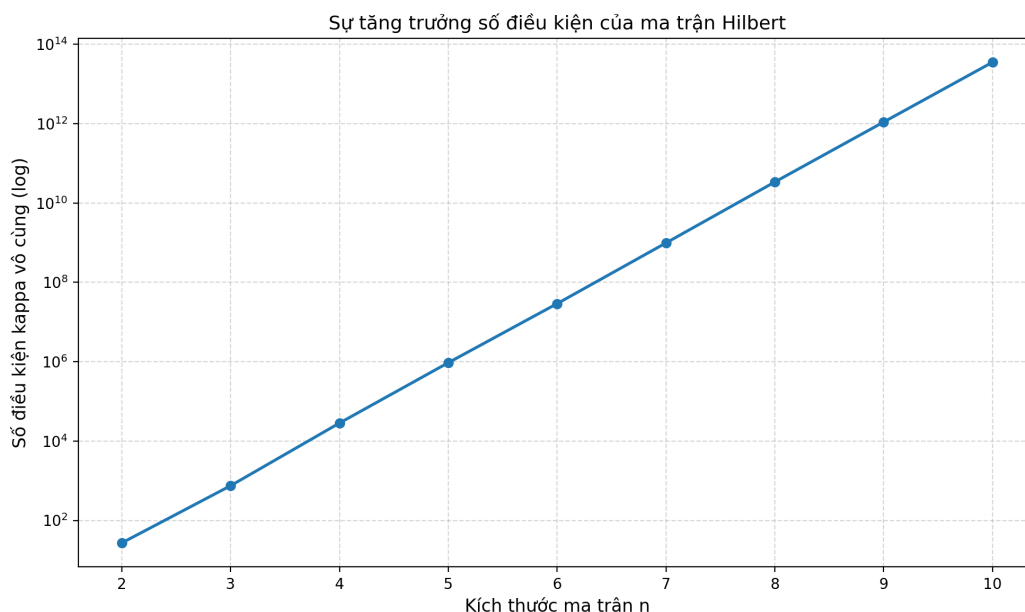
Nhóm đối chiếu sai số dư của hai dạng ma trận vuông với kích thước cơ sở $n = 10$:

- **Ma trận Hilbert (H_n):** Khởi tạo dưới dạng $H_{ij} = \frac{1}{i+j-1}$. Ma trận này được biết đến với cấu trúc đặc trưng thiếu điều kiện số học (ill-conditioned) do khoảng cách giữa các vector cột vô cùng gần nhau.
- **Ma trận Đối xứng Xác Định Dương (SPD):** Khởi tạo bằng phép nhân $A = X^T X + nI$ để bảo đảm tính xác định dương.

Đối với Gauss–Seidel, điều kiện hội tụ đủ là chéo trội; tuy nhiên với ma trận SPD, phương pháp vẫn hội tụ theo lý thuyết, dù không nhất thiết thỏa chéo trội theo hàng. Trong thí nghiệm này, số điều kiện được trình bày theo $\kappa(A)$ như trong notebook thực nghiệm.

Bảng 7: Sai số dư và số điều kiện (chuẩn vô cùng) với $N = 10$

Loại ma trận	Số điều kiện $\kappa(A)$	Sai số dư Gauss	Sai số dư QR	Sai số dư GS
Hilbert (Ill-conditioned)	1.60×10^{13}	0.00	2.20×10^{-6}	4.19×10^{-6}
SPD (Well-conditioned)	2.20	1.96×10^{-16}	2.10×10^{-16}	1.22×10^{-11}



Hình 5: Biểu đồ phân tích độ khuếch đại của số điều kiện Hilbert khi kích thước ma trận n gia tăng

3.4.2. Phân tích sai số theo số điều kiện

Ma trận Hilbert có $\kappa(A)$ rất lớn ($\approx 1.6 \times 10^{13}$), dẫn đến sai số dư của QR và Gauss–Seidel ở mức 10^{-6} . Trong khi đó, ma trận SPD có $\kappa(A)$ nhỏ (≈ 2.2), nên cả Gauss và QR đều đạt sai số dư gần 10^{-16} . Kết quả này phù hợp với lý thuyết sai số: khi $\kappa(A)$ lớn, sai số bị khuếch đại đáng kể ngay cả khi thuật toán ổn định.

Ở trường hợp SPD, Gauss–Seidel vẫn cho sai số dư lớn hơn hai phương pháp trực tiếp. Nguyên nhân chính là đây là phương pháp lặp với ngưỡng dừng hữu hạn, nên sai số dư không thể tiệm cận máy chính xác như Gauss và QR.

3.4.3. Kết luận về tính ổn định của thuật toán

Kết quả thực nghiệm cho thấy việc **kiểm định số điều kiện** là bước bắt buộc trước khi giải hệ tuyến tính. Các ma trận ill-conditioned như Hilbert có thể làm mất độ tin cậy của nghiệm ngay cả với kích thước nhỏ. Điều này khẳng định rằng tối ưu hóa mã nguồn không thể thay thế cho việc đánh giá điều kiện của bài toán.

3.5. Chỉ dẫn lựa chọn giải thuật

Dựa trên sự cân bằng giữa chi phí thực thi và độ ổn định số học, nhóm đề xuất quy trình lựa chọn thuật toán giải hệ $Ax = b$ như sau:

- **Phương pháp Khử (Gauss / LU / Cholesky):** Ưu tiên sử dụng cho các ma trận đặc (Dense Matrices) có kích thước từ nhỏ đến trung bình (dưới 10.000 ẩn số) và có số điều kiện thấp ($\kappa(A) < 10^3$). Đây là lựa chọn tối ưu về mặt tốc độ trong điều kiện ma trận ổn định.
- **Phương pháp Trực giao (QR):** Áp dụng cho các bài toán yêu cầu độ ổn định số học cao hoặc các hệ phương trình không vuông (mô hình hồi quy, Bình phương tối thiểu). Mặc dù chi phí thời gian cao hơn, QR đảm bảo tính chính xác cho các hệ có nguy cơ nhiều số học.
- **Phương pháp Lặp (Gauss-Seidel):** Là lựa chọn hàng đầu cho các ma trận thưa quy mô lớn (Sparse Matrices với tỷ lệ phần tử khác 0 thấp), ma trận đường chéo trội hoặc các hệ phương trình đạo hàm riêng (PDEs). Phương pháp này tối ưu hóa việc sử dụng bộ nhớ và thời gian tính toán thông qua cơ chế hội tụ dần.

KẾT LUẬN

1. Các kết quả đạt được

Sau quá trình nghiên cứu, thiết kế thuật toán và lập trình thực nghiệm, nhóm đã hoàn thành các mục tiêu đề ra của Đồ án môn Toán Ứng dụng và Thống kê. Cụ thể:

- **Về mặt Lập trình & Thuật toán:** Cài đặt thành công các hàm từ đầu bằng Python thuần. Cài đặt thuật toán Khử Gauss có chọn phần tử chốt (Partial Pivoting) giúp đảm bảo độ chính xác cao khi giải hệ phương trình và tìm nghịch đảo. Tại phần phân rã, nhóm đã triển khai thành công thuật toán trực giao hóa Gram-Schmidt cổ điển và thuật toán Lặp QR (QR Iteration) để tìm giá trị riêng, vector riêng và chéo hóa ma trận.
- **Về mặt Đánh giá & Kiểm thử:** Xây dựng được hệ thống testcase bao phủ các trường hợp từ ma trận lý tưởng, ma trận suy biến, đến các ma trận số học như ma trận Hilbert hay ma trận chênh lệch tỷ lệ cực đoan (Extreme Scale).
- **Về mặt Trực quan hóa:** Render thành công video 3D bằng Manim mô phỏng trực quan từng bước chiếu vector của thuật toán Gram-Schmidt và hiệu ứng tạo lập không gian tọa độ mới trong phép biến đổi $A = PDP^{-1}$.

2. Khó khăn gặp phải

Trong quá trình thực hiện đồ án, nhóm đã đối mặt và giải quyết nhiều thách thức khó khăn, đặc biệt là ở mảng Tính toán số (Numerical Computing):

- **Sai số làm tròn (Round-off Errors) trong Gram-Schmidt:** Thuật toán Gram-Schmidt cổ điển thể hiện sự bất ổn định số học nghiêm trọng khi gặp các vector gần phụ thuộc tuyến tính (Numerically Dependent). Hiện tượng triệt tiêu sai số (Catastrophic Cancellation) khiến ma trận Q nhanh chóng mất đi tính trực giao hoàn hảo.
- **Tốc độ hội tụ của thuật toán Lặp QR:** Do giới hạn yêu cầu tự cài đặt, thuật toán Lặp QR thuần túy (không có cơ chế dịch chuyển Shifts) mất rất nhiều thời gian (hàng trăm vòng lặp) mới có thể hội tụ khi gặp các trị riêng nằm quá sát nhau.
- **Trực quan hóa không gian 3D:** Thư viện Manim có learning-curve (đường cong học tập) khá dốc. Việc đồng bộ hóa các phép chiếu toán học với hoạt ảnh (animations) của camera 3D tốn rất nhiều thời gian canh chỉnh tọa độ.

- **Kiến thức khó:** Đồ án có nhiều phần kiến thức khó, đòi hỏi phải nghiên cứu nhiều tài liệu tham khảo và tham khảo nhiều nguồn trên Internet, khó khăn nhất là cài các thuật toán mà không sử dụng thư viện Numpy.

3. Bài học rút ra

Sau quá trình cùng nhau hoàn thành đồ án thì nhóm có rút ra được một số bài học cho mình:

1. **Khoảng cách giữa lý thuyết và thực hành:** Một công thức toán học đúng hoàn toàn trên giấy (như Gram-Schmidt) chưa chắc đã là một thuật toán tốt khi đưa vào bộ nhớ dấu phẩy động (Float 64-bit) của máy tính. Việc hiểu rõ giới hạn phần cứng giúp nhóm có tư duy lập trình phòng thủ tốt hơn.
2. **Hiểu sâu hơn về các thư viện công nghiệp:** Qua việc tự cài đặt và liên tục nhận kết quả thua kém so với NumPy ở các test cases khó, nhóm đã hiểu được tại sao các thư viện hiện đại phải sử dụng những thuật toán phức tạp hơn rất nhiều như Householder Reflections (để phân rã) và Wilkinson Shifts (để chéo hóa).
3. **Kỹ năng:** Qua quá trình cùng nhau làm và nghiên cứu các tài liệu, từng thành viên trong nhóm đồng thời phát triển nhiều kỹ năng mềm như: research, làm việc nhóm, giao tiếp. Các kỹ năng trên đều là kỹ năng nền tảng quan trọng cho tương lai sau này.

TÀI LIỆU THAM KHẢO

- [1] G. Strang, *Introduction to Linear Algebra*, 6th. Wellesley-Cambridge Press, 2023.
- [2] L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*. SIAM, 1997.
- [3] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th. Johns Hopkins University Press, 2013.
- [4] NumPy Developers. “Numpy reference documentation - linear algebra (numpy.linalg),” Accessed: May 10, 2024. [Online]. Available: <https://numpy.org/doc/stable/reference/routines.linalg.html>
- [5] Manim Community Developers. “Manim community documentation - mathematical animation framework,” Accessed: May 10, 2024. [Online]. Available: <https://docs.manim.community/>

PHỤ LỤC

Danh sách Link Video Demo

Dưới đây là các đường dẫn video trực quan hóa thuật toán bằng thư viện Manim và các kết quả demo của nhóm:

- **Video 1 (Minh họa Phép khử Gauss):** <https://youtube.com/...>
- **Video 2 (Trực quan hóa Phân rã QR):** <https://youtube.com/...>
- **Video 3 (Chéo hóa ma trận):** <https://youtube.com/...>